

How Do Static and Dynamic Test Case Prioritization Techniques Perform on Modern Software Systems? An Extensive Study on GitHub Projects

Qi Luo^{ID}, *Student Member, IEEE*, Kevin Moran, *Student Member, IEEE*,
Lingming Zhang, *Member, IEEE*, and Denys Poshyvanyk^{ID}, *Member, IEEE*

Abstract—Test Case Prioritization (TCP) is an increasingly important regression testing technique for reordering test cases according to a pre-defined goal, particularly as agile practices gain adoption. To better understand these techniques, we perform the first extensive study aimed at empirically evaluating four static TCP techniques, comparing them with state-of-research dynamic TCP techniques across several quality metrics. This study was performed on 58 real-world Java programs encompassing 714 KLoC and results in several notable observations. First, our results across two effectiveness metrics (the Average Percentage of Faults Detected *APFD* and the cost cognizant *APFDc*) illustrate that at test-class granularity, these metrics tend to correlate, but this correlation does not hold at test-method granularity. Second, our analysis shows that static techniques can be surprisingly effective, particularly when measured by *APFDc*. Third, we found that TCP techniques tend to perform better on larger programs, but that program size does not affect comparative performance measures between techniques. Fourth, software evolution does not significantly impact comparative performance results between TCP techniques. Fifth, neither the number nor type of mutants utilized dramatically impact measures of TCP effectiveness under typical experimental settings. Finally, our similarity analysis illustrates that highly prioritized test cases tend to uncover dissimilar faults.

Index Terms—Regression testing, test case prioritization, static, dynamic, mutation analysis

1 INTRODUCTION

MODERN software evolves at a constant and rapid pace; developers continually add new features and fix bugs to ensure a satisfied user base. During this evolutionary process, it is crucial that developers do not introduce new bugs, known as software *regressions*. *Regression testing* is a methodology for efficiently and effectively validating software changes against an existing test suite aimed at detecting such bugs [46], [74]. One of the key tasks of the contemporary practice of continuous regression testing, is test case prioritization (TCP).

Regression test prioritization techniques reorder test executions in order to maximize a certain objective function, such as exposing faults earlier or reducing the execution time cost [46]. This practice can be readily observed in applications to large industrial codebases such as at Microsoft,

where researchers have built test prioritization systems for development and maintenance of Windows for a decade [11], [61]. In academia, there exists a large body of research that studies the design and evaluate regression TCP techniques [41], [46], [56], [57], [67], [73]. Traditionally, TCP techniques leverage one of several code coverage measurements of tests from a previous software version as a representation of test effectiveness on a more recent version. These approaches use this coverage-based test adequacy criterion to iteratively compute each test's priority, and then rank them to generate a prioritized list. Researchers have proposed various forms of this traditional approach to TCP, including greedy (total and additional strategies) [56], [57], [73], adaptive random testing [32], and search-based strategies [45].

While dynamic TCP techniques can be useful in practice, they may not be always applicable due to certain notable shortcomings, including: 1) the time cost of executing an instrumented program to collect coverage information [25], [50]; 2) expensive storage and maintenance of coverage information [50], [76]; 3) imprecise coverage metrics due to code changes during evolution or thread scheduling of concurrent systems [44], and 4) the absence of coverage information for newly added tests [46] or systems/modules that disallow code instrumentation [44] (e.g., code instrumentation may break the time constraints of real-time systems). Thus, to offer alternative solutions that that do not exhibit

- Q. Luo, K. Moran, and D. Poshyvanyk are with the Department of Computer Science, College of William and Mary, Williamsburg, VA 23188. E-mail: {qluo, kpmoran, denys}@cs.wm.edu.
- L. Zhang is with the Department of Computer Science, University of Texas at Dallas, Dallas, TX 75080. E-mail: lingming.zhang@utdallas.edu.

Manuscript received 9 May 2017; revised 10 Feb. 2018; accepted 20 Mar. 2018. Date of publication 2 Apr. 2018; date of current version 12 Nov. 2019. (Corresponding author: Qi Luo.)

Recommended for acceptance by G. Fraser.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TSE.2018.2822270

many of these shortcomings, researchers have proposed a number of TCP techniques that rely solely upon *static* information extracted from the text of source and test code. Unfortunately, since the introduction of purely static TCP techniques, little research has been conducted to fully investigate the effectiveness of static techniques on modern software. This begs several important questions in the context of past work on dynamic techniques, such as: *How does the effectiveness of static and dynamic techniques compare on modern software projects? Do static and dynamic techniques uncover similar faults? How efficient are static techniques when compared to one another?* The answers to these questions are of paramount importance as they will guide future research directions related to TCP techniques.

Several empirical studies have been conducted in an attempt to examine and understand varying aspects of different TCP approaches [16], [21], [55], [56], [63]. However, there are clear limitations of prior studies that warrant further experimental work on TCP techniques: 1) recently proposed TCP techniques, particularly static techniques, have not been thoroughly evaluated against each other or against techniques that operate upon dynamic coverage information; 2) no previous study examining static TCP approaches has comprehensively examined the impact of different test granularities (e.g., prioritizing entire test classes or individual test methods), the efficiency of the techniques, or the similarities in terms of uncovered faults; 3) prior studies have typically failed to investigate the application of TCP techniques to sizable real-world software projects, and none of them have investigated the potential impact of program size (i.e., LOC) on the effectiveness of TCP techniques; 4) prior studies have not comprehensively investigated the impact of the quantities of faults used to evaluate TCP approaches; and 5) no previous study has attempted to gain an understanding of the impact of fault characteristics on TCP evaluations.

Each of these points are important considerations that call for thorough empirical investigation. For instance, studying the effectiveness and similarity of faults uncovered for *both* static and dynamic techniques could help inform researchers of potential opportunities to design more effective and robust TCP approaches. Additionally, evaluating a set of popular TCP techniques on a large group of sizable real-world java programs would help bolster the generalizability of performance results for these techniques. Another important consideration that arises from limitations of past studies is that an increasing number of studies use mutants as a proxy for real faults to evaluate performance characteristics of TCP techniques. Thus, understanding the effect that mutant quantities and operators have on mutation analysis-based TCP evaluations should help researchers design more effective and reliable experiments, or validate existing experimental settings for continued use in future work. Therefore, in this paper we evaluate the effectiveness of TCP approaches in terms of detecting mutants.

To answer the unresolved questions related to the understanding of TCP techniques and address the current gap in the existing body of TCP research we perform an extensive empirical study comparing four popular static TCP techniques, i.e., call-graph-based (with total and additional

strategies) [76], string-distance-based [43], and topic-model based techniques [63] to four state-of-the-art dynamic TCP techniques (i.e., the greedy-total [56], greedy-additional [56], adaptive random [32], and search-based techniques [45]) on 58 real-world software systems. All of the studied TCP techniques were implemented based on the papers that initially proposed them and the implementation details are explained in Section 3.4. It is important to note that different granularities of dynamic coverage information may impact the effectiveness of dynamic TCP techniques. In this paper, we examine statement-level coverage for dynamic techniques, since previous work [46], [50] has illustrated that statement-level coverage is *at least as effective* as other common coverage criteria (e.g., method and branch coverage) in the TCP domain. In our evaluation criteria we examine the effectiveness of the studied techniques in terms of the Average Percentage of Faults Detected (APFD) and its cost cognizant version APFDc. Additionally, we analyze the implications of these two metrics as efficacy measures of TCP techniques and discuss the implications of this analysis. We also analyze the impact of subject size and software evolution on these metrics. Furthermore, during our empirical study, we vary the operator types and the quantities of injected mutants to investigate whether these factors significantly affect the evaluation of TCP approaches. We also examine the similarity of detected faults for the resultant prioritized sets of test cases generated by our studied TCP techniques at different test granularities (e.g., both method and class levels). More specifically, we investigate the total number and the relative percentages of different types of mutants detected by the most highly prioritized test cases for each TCP technique to further understand their capabilities in detecting faults with varying attributes. Finally, we examine the efficiency, in terms of execution time (i.e., the processing time for TCP technique), of static TCPs to better understand the time cost associated with running these approaches.

Our study bears several notable findings. The first of these is that there are statistically significant differences among the APFD values of all studied techniques. When measuring the average APFD values across our subject programs, we found that the call-graph-based (with “additional” strategy) technique outperforms all studied techniques at the test-class level. At the test-method level, the call-graph and topic-model based techniques perform better than other static techniques, but worse than two dynamic techniques, i.e., the additional and search-based techniques. *Second, our results demonstrate that APFDc values are generally consistent with APFD values at test-class level but relatively less consistent at test-method level.* When examining the effectiveness of TCP approaches in terms of the cost-cognizant APFDc values, we found that the call-graph-based (with “additional” strategy) technique outperforms all studied dynamic and static techniques at both test-class and test-method levels, indicating the limitations of dynamic execution information in reducing actual regression testing time costs. Additionally, while APFDc values vary dramatically across 58 subject programs, based on the results of our analysis, there are no statistically significant differences between TCP techniques based on APFDc values at both of test-class and test-method level when controlling for the subject program. *Third, our results*

TABLE 1
The List of Contributions

Contributions	Descriptions
Static versus Dynamic TCP	To the best of the authors' knowledge, this is the first extensive empirical study that compares the effectiveness, efficiency, and similarity of uncovered faults of both static <i>and</i> dynamic TCP techniques on a large set of modern real-world programs;
Impact of Performance Metrics	We evaluate the performance of TCP techniques based on two popular metrics, APFD and APFDc, and understand the relationship between the performance of these two metrics for TCP evaluation;
Impact of Test Case Granularity	We evaluate the performance of TCP techniques at two different test granularities, and investigate the impacts of test granularities on TCP evaluation;
Impact of Program Subject Size	We evaluate the impacts of subject size on the effectiveness of the studied static and dynamic TCP techniques;
Impact of Software Evolution	We evaluate the impacts of software evolution on the effectiveness of the studied static and dynamic TCP techniques;
Impact of the Number of Studied Faults	We conduct the first study investigating the impact of different fault quantities used in the evaluation on the effectiveness of TCP techniques;
Impact of Fault Types	We conduct the first study investigating the impact of different fault types used in the evaluation on the effectiveness of TCP techniques;
Analysis of Similarity of Uncovered Faults	We analyze the similarity of faults uncovered between prioritized sets of test cases generated by different techniques;
Practical Guidelines for Future Research	We discuss the relevance and potential impact of the findings in the study, and provide a set of learned lessons to help guide future research in TCP;
Open Source Dataset	We provide a publicly available, extensive online appendix and dataset of the results of this study to ensure reproducibility and aid future research [48].

indicate that the test granularity dramatically impacts the effectiveness of TCP techniques. While nearly all techniques perform better at method-level granularity based on both of APFD and APFDc values, the static techniques perform comparatively worse to dynamic techniques at method level as opposed to class level based on APFD values. Fourth, our study shows that differences in the subject size, software version, mutant types, or mutant quantities tend not to largely impact experimental results measuring TCP performance. In terms of execution time, call-graph based techniques are the most efficient of the static TCP techniques. Finally, the results of our similarity analysis study suggest that there is minimal overlap between the uncovered faults of the studied dynamic and static TCPs, with the top 10 percent of prioritized test-cases only sharing $\approx 25 - 30$ percent of uncovered faults. Thus, the most highly prioritized test cases from different TCP techniques exhibit dissimilar capabilities in detecting different types of mutants. This suggests that certain TCP techniques may be better at uncovering faults (or mutants) that exhibit certain characteristics, and that aspects of different TCP techniques may be combined together to alter performance characteristics. Both of these findings are promising avenues for future work. To summarize, this paper makes the following noteworthy contributions summarized in Table 1.

2 BACKGROUND & RELATED WORK

In this section we formally define the TCP problem, introduce our studied set of subject studied techniques, and further differentiate the novelty and research gap that our study fulfills. Rothermel et al. [57] formally defined the test prioritization problem as finding $T' \in P(T)$, such that $\forall T'', T'' \in P(T) \wedge T'' \neq T' \Rightarrow f(T') \geq f(T'')$, where $P(T)$ denotes the set of permutations of a given test suite T , and f denotes a function from $P(T)$ to real numbers. All studied techniques attempt to address the TCP problem formally enumerated above with the objective function of uncovering the highest number of faults with the smallest set of most highly prioritized test cases. As defined in previous work [28], [63], a white-box TCP approach requires access to *both* the source code of subject programs, and other types of information (e.g., test code), whereas black-box techniques do not require the source code or test code of subject programs, and grey-box techniques require access to only the test-code. Most dynamic techniques (including the ones considered in this study) are considered white-box techniques since they require access to the subject system's source code. In our study, we limit our focus to white and grey-box static TCP techniques that require only source code and test cases, and the dynamic TCP techniques that only require dynamic coverage and test cases as inputs for two main reasons: 1) this represents fair comparison of similar techniques that leverage traditional inputs (e.g., test cases, source code and coverage info), and 2) the inputs needed by other techniques (e.g., requirements, code changes, user knowledge) are not always available in real-world subject programs. In the next two subsections, we introduce the underlying methodology utilized by our studied static TCP techniques (Section 2.1) and dynamic TCP techniques (Section 2.2). Details of our own reimplementations of these tools are discussed later in Section 3. Additionally, we discuss existing empirical studies (Section 2.3).

2.1 Static TCP Techniques

Call-Graph-Based. This technique builds a call graph for each test case to obtain a set of transitively invoked methods, called *relevant methods* [76]. The test cases with a higher number of invoked methods in the corresponding call-graphs are assigned a higher test ability and thus are prioritized first. This approach is often implemented as one of two variant two sub-strategies, the *total* strategy prioritizes the test cases with higher test abilities earlier, and the *additional* strategy prioritizes the test cases with higher test abilities excluding the methods that have already been covered by the prioritized test cases. Further research by Mei et al. extends this work to measure the test abilities of the test cases according to the number of invoked statements as opposed to the number of invoked methods [50]. The main intuition behind such an extension is that by allowing for a more granular representation of test ability (at the statement level) leads to a more effective overall prioritization scheme. This call-graph based technique is classified as a white box approach, whereas the other two studied static TCP techniques are grey-box approaches, requiring only test code. We consider both types of static techniques in this paper in order to thoroughly compare them to a set of techniques that require dynamic computation of coverage.

String-Distance-Based. The key idea underlying this technique is that test cases that are textually different from one another, as measured by similarity based on string-edit distance, should be prioritized earlier [43]. The intuition behind this idea is that textually dissimilar test cases have a higher probability of executing different paths within a program. This technique is a *grey-box* static technique since the only information it requires is the test code. There are four major variants of this technique differentiated by the string-distance metric utilized to calculate the gap between each pair of test cases: Hamming, Levenshtein, Cartesian, and Manhattan distances. Based on prior experimental results [43], Manhattan distance performs best in terms of detecting faults. Thus, in our study, we implemented the string-based TCP based on the paper by Ledru et al. [43], and chose Manhattan distance as the representative string distance computation for this technique. Explicit details regarding our implementation are given in Section 3.

Topic-Based. This static black-box technique further abstracts the concept of using test case diversity for prioritization by utilizing semantic-level topic models to represent tests of differing functionality, and gives higher prioritization to test cases that contain different topics from those already executed [63]. The intuition behind this technique is that semantic topics, which abstract test cases' functionality, can capture more information than simple textual similarity metrics, and are robust in terms of accurately differentiating between dissimilar test cases. This technique constructs a vector based on the code of each test case, including the test case's correlation values with each semantically derived topic. It calculates the distances between these text case vectors using a Manhattan distance measure, and defines the distance between one test case and a set of test cases as the minimum distance between this test case and all test cases in the set. During the prioritization process, the test case which is farthest from all other test cases is first selected and put into the (originally empty) prioritized set. Then, the technique iteratively adds the test case farthest from the prioritized set into the prioritized set until all tests have been added.

Other Approaches. In related literature, researchers have proposed various other techniques to prioritize tests based on software requirement documents [6] or system models [40]. Recently, Saha et al. proposed an approach that uses software trace links between source code changes and test code derived via Information Retrieval (IR) techniques and sorts the test cases based according to the relationships inferred via the trace links, with tests more closely corresponding to changes being prioritized first [58]. These techniques require additional information, such as the requirement documents, system models, and code changes, which may be unavailable or challenging to collect. In this study, we center our focus on automated TCP techniques that require only the source code and the test code of subjects, including call-graph-based, string-based and topic-based techniques.

2.2 Dynamic TCP Techniques

Greedy Techniques. As explained in our overview of the Call-Graph-based approach, there are typically two variants of traditional "greedy" dynamic TCP techniques, the *total* strategy and *additional* strategy, that prioritize test cases based on code coverage information. The total strategy

prioritizes test cases based on their absolute code coverage, whereas the additional strategy prioritizes test cases based on each test case's contribution to the total cumulative code coverage. In our study, we implemented these techniques based on prior work by Rothermel et al. [56]. The greedy-*additional* strategy has been widely considered as one of the most effective TCP techniques in previous work [32], [73]. Recently, Zhang et al. proposed a novel approach to bridge the gap between the two greedy variants by unifying the strategies based on the fault detection probability [27], [73].

Given that these dynamic TCP techniques utilize code coverage information as a proxy for test effectiveness, and many different coverage metrics exist, studies have examined several of these metrics in the domain of TCP including statement coverage [56], basic block and method coverage [16], Fault-Exposing-Potential (FEP) coverage [21], transition and round-trip coverage [70]. For instance, Do et al. use both method and basic block coverage information to prioritize test cases [16]. Elbaum et al. proposed an approach that prioritizes test cases based on their FEP and fault index coverage [21], where test cases exposing more potential faults will be assigned a higher priority. Kapfhammer et al. use software requirement coverage to measure the test abilities of test cases for test prioritization [36].

Adaptive Random Testing. Jiang et al. were the first to apply Adaptive Random Testing [10] to TCP and proposed a novel approach, called Adaptive Random Test Case Prioritization (ART) [32]. ART randomly selects a set of test cases iteratively to build a candidate set, then it selects from the candidate set the test case farthest away from the prioritized set. The whole process is repeated until all test cases have been selected. As a measure of distances between test cases, ART first calculates the distance between each pair of test cases using Jaccard distance based on their coverage, and then calculates the distance between each candidate test case and the prioritized set. Three different variants of this approach exist (*min*, *avg* and *max*), differentiated by the type of distance used to determine the similarity between one test case and the prioritized set. For example, *min* is the minimum distance between the test case and the prioritized test case. The results from Jiang et al.'s evaluation illustrates that ART with *min* distance performs best for TCP. Thus, in our empirical study, we implemented our ART based TCP strategy following Jiang et al.'s paper [32] and chose *min* distance to estimate the distance between one test case and the prioritized set.

Search-Based Techniques. Search-based TCP techniques introduce meta-heuristic search algorithms into the TCP domain, exploring the state space of test case combinations to find the ranked list of test cases that detect faults more quickly [45]. Li et al. have proposed two variants of search-based TCP techniques, based upon hill-climbing and genetic algorithms. The hill-climbing-based technique evaluates all neighboring test cases in a given state space, locally searching the ones that can achieve largest increase in fitness. The genetic technique utilizes an evolutionary algorithm that halts evolution when a predefined termination condition is met, e.g., the fitness function value reaches a given value or a maximal number of iterations has been reached. In our empirical study, we examine the genetic-based test prioritization approach as the representative search-based test case prioritization technique, as previous

results demonstrate that genetic-based technique is more effective in detecting faults [45].

Other Approaches. Several other techniques that utilize dynamic program information have been proposed, but do not fit neatly into our classification system enumerated above [30], [51], [64]. Islam et al. presented an approach that reconciles information from traceability links between system requirements and test cases and dynamic information, such as execution cost and code coverage, to prioritize test cases [30]. Nguyen et al. have designed an approach that uses IR techniques to recover the traceability links between change descriptions and execution traces for test cases to identify the most relevant test cases for each change description [51]. Unfortunately, these TCP techniques require information beyond the test code and source code (e.g., execution cost, user knowledge, code changes) which may not be available or well maintained depending on the target software project. In this paper, we choose dynamic techniques that require only code coverage and test cases for comparison, which includes three techniques (i.e., Greedy (with total, additional strategies), ART, and Search-based). Recall that we do not aim to study the impact of coverage granularity on the effectiveness of dynamic TCPs, and opt to utilize only statement level coverage information in our experiments. This is because previous work has established that statement-level coverage is *at least as effective* as other coverage types [46], [50].

2.3 Empirical Studies on TCP Techniques

Several studies empirically evaluating TCP techniques [8], [14], [21], [22], [23], [23], [28], [37], [46], [55], [56], [60], [69], [71], [72] have been published. In this section we discuss the details of the studies most closely related to our own in order to illustrate the novelty of our work and research gap filled by our proposed study. Rothmel et al. conducted a study on unordered, random, and dynamic TCP techniques (e.g., coverage based, FEP-based) applied to C programs, to evaluate their abilities of fault detection [56]. Elbaum et al. conducted a study on several dynamic TCP techniques applied to C programs in order to evaluate the impact of software evolution, program type, and code granularity on the effectiveness of TCP techniques [21]. Thomas et al. [63] compared the topic-based TCP technique to the static string-based, call-graph-based techniques, and greedy-additional dynamic technique at method-level on two subjects. However, this study is limited by a small set of subject programs, a comparison to only one dynamic technique at method-level only, and no investigation of fault detection similarity, the effects of software evolution or subject program size among the approaches.

Do et al. have presented a study of dynamic test prioritization techniques (e.g., random, optimal, coverage-based) on four Java programs with JUnit test suites. This study breaks from past studies that utilize only small C programs and demonstrates that these techniques can also be effective on Java programs. However, findings from this study also suggest that different languages and testing paradigms may lead to divergent behaviors [16]. This group also conducted an empirical study to analyze the effects of time constraints on TCP techniques [12]. Henard et al. recently conducted a study comparing white and black-box TCP techniques in which the effectiveness, similarity, efficiency, and

performance degradation of several techniques was evaluated. While this is one of the most complete studies in terms of evaluation depth, it does not consider the static techniques considered in this paper. Thus, our study is differentiated by the unique goal of understanding the relationships between purely static and dynamic TCPs.

To summarize, while each of these studies offers valuable insights, none of them provides an in-depth evaluation and analysis of the effectiveness, efficiency, and similarity of detected faults for static TCP techniques and comparison to dynamic TCP techniques on a set of mature open source software systems. This highlights a clear research gap that exists in prior work that conduct empirical studies measuring the efficacy of TCP techniques. The work conducted in this paper is meant to close this gap, and offer researchers and practitioners an extensive, rigorous evaluation of popular TCP techniques according to extensive set of metrics and experimental investigations.

2.4 Mutation Analysis

Fault detection effectiveness is almost universally accepted as the measurement by which to evaluate TCP approaches [3], [33], [46]. However, extracting a suitable set of representative real-world faults is typically prohibitively costly. Thus, researchers and developers commonly evaluate the effectiveness of TCP approaches using mutation analysis, in which a set of program variants, called mutants, are generated by seeding a large number of small syntactic errors into a seemingly “correct” version of a program. For a given subject program, mutation operators are utilized to seed these faults (known as *mutants*) into an unmodified version of the program. It is said that a mutant is *killed* by a test case when this test case is able to detect a difference between the unmodified program and the mutant. In the context of TCP research, mutation analysis is applied to a subject program to generate a large set of mutants, each containing a minor fault, and then this set is used to evaluate the effectiveness of a set of prioritized test cases.

Preliminary studies have shown mutants to be suitable for simulating real bugs in software testing experiments in controlled contexts [4], [33], and mutation analysis has been used to evaluate many different types of testing approaches, including TCP techniques [21], [28], [46], [63], [68]. For example, Henard et al. utilized mutation analysis to compare white-box and black-box TCP techniques [28]. Lu et al. evaluated the test case prioritization techniques in the context of evolving software systems using mutation analysis [46]. Finally, Walcott et al. proposed a time-aware test prioritization technique and evaluated their approach using mutants [68].

Additionally, recent research has been undertaken that aims to understand the relationship between different *types* of mutants (e.g., operators) and whether or not they are a suitable proxy for *real* faults [2], [34], [35], [39]. Ammann et al. proposed a framework to reduce redundant mutants and determine a minimal set of mutants for properly evaluating test cases [2]. Kintis et al. introduced several alternatives to mutation testing strategies to establish whether they adversely affect measuring test effectiveness [39]. However, previous studies do not provide comments on the following in the context of TCP: 1) none of these studies has

investigated the impact of the quantity of mutants utilized in TCP experiments; and 2) previous work has not examined the impact of mutants seeded according to different operators on the effectiveness of TCP approaches. It is quite possible that TCP may perform differently when detecting different quantities or types of mutants, particularly across software projects. Addressing these current shortcomings of past studies would allow for the verification or refutation of previous widely used experimental settings for mutation-based TCP evaluations. Thus, we aim to evaluate the effectiveness of TCP techniques in terms of detecting different quantities and types of mutants in order to understand their impact on this quality metric.

2.5 Metrics for TCP Techniques

The Average Percentage of Faults Detected (APFD) metric is a well-accepted metric in the TCP domain [15], [17], [18], [21], [56], [57], [76], which is used to measure the effectiveness, in terms of fault detection rate, for each studied test prioritization technique. Formally speaking, let T be a test suite and T' be a permutation of T , the APFD metric for T' is computed according to the following metric:

$$APFD = 1 - \frac{\sum_{i=1}^m TF_i}{n * m} + \frac{1}{2n}, \quad (1)$$

where n is the number of test cases in T , m is the number of faults, and TF_i is the position of the first test case in T' that detects fault i .

Although APFD has been widely used for evaluating TCP techniques, it assumes that each test incur the same time cost, an assumption which often doesn't hold up in practice. Thus, Elbaum et al. introduced another metric, called APFDc [19]. APFDc is the cost-cognizant version of APFD, which considers both the test case execution cost and fault severity. While not as widely used as APFD, APFDc has also been used to evaluate TCP approaches, resulting in a more detailed evaluation. [24]. APFDc can be formally defined as follows: let $t_1, t_2, \dots, t_n$ be the execution costs for all the n test cases. and $f_1, f_2, \dots, f_m$ be the severities of the m detected faults. The APFDc metric is calculated according to the following equation:

$$APFDc = \frac{\sum_{i=1}^m f_i * (\sum_{j=TF_i}^n t_j - \frac{1}{2} t_{TF_i})}{\sum_{i=1}^n t_i * \sum_{i=1}^m f_i}. \quad (2)$$

Similar to Equation (1), n is the number of test cases in T , m is the number of faults, and TF_i is the position of the first test case in T' that detects fault i . In our empirical study, we evaluate the performance of TCP techniques based on both of APFD and APFDc, in order to provide a complete picture of the performance of TCP techniques from the perspective of both effectiveness and efficiency. Additionally, we further examine the relationship between these two metrics and the resultant implications for the domain of TCP research.

3 EMPIRICAL STUDY

In this section, we state our research questions, and enumerate the subject programs, test suites, study design, and implementation of studied techniques in detail.

3.1 Research Questions (RQs):

Our empirical study addresses the following RQs:

- RQ₁** How do static TCP techniques compare with each other and with dynamic techniques in terms of *effectiveness* measured by APFD?
- RQ₂** How do static TCP techniques compare with each other and with dynamic techniques in terms of *effectiveness* measured by APFDc?
- RQ₃** How does the *test granularity* impact the effectiveness of both the static and dynamic TCP techniques?
- RQ₄** How does the *program size* (i.e., LOC) impact the effectiveness of both the static and dynamic TCP techniques?
- RQ₅** How do static and dynamic TCP techniques perform *as software evolves*?
- RQ₆** How does the *quantity of mutants* impact the effectiveness of the studied TCP techniques?
- RQ₇** How does *mutant type* impact the effectiveness of the studied TCP techniques?
- RQ₈** How *similar* are different TCP techniques in terms of detected faults?
- RQ₉** How does the *efficiency* of static techniques compare with one another in terms of execution time cost?

To aid in answering RQ₁ and RQ₂, we introduce the following null and alternative hypotheses. The hypotheses are evaluated at the 0.05 level of significance:

- H_0 : There is no statistically significant difference in the effectiveness between the studied TCPs.
- H_a : There is a statistically significant difference in the effectiveness between the studied TCPs.

3.2 Subject Programs, Test Suites and Faults

We conduct our study on 58 real-world Java programs from GitHub[1]. The program names and sizes in terms of lines of code (LOC) are shown in Table 2, where the sizes of subjects vary from 1,151 to 82,998 LoC. Our subjects are larger in size and quantity than previous work in the TCP domain [28], [32], [43], [46], [63]. Our methodology for collecting these subject programs is as follows. We first collect a set of 399 Java programs from GitHub that contain integrated JUnit test cases and can be compiled successfully. Then, we discarded programs which were relatively small in size (i.e., less than 1,000 LOC), or that had very small numbers of test cases (i.e., less than 15 test cases at method level and five test cases at class level). Finally, we ran a set of tools to collect both the static and dynamic information (Section 3.4) and discarded programs for which the tools were not applicable. After this process we obtained our set of 58 subject programs.

To perform this study, we checked out the most current master branch of each program, and provide the version IDs in our online appendix [48]. For each program, we used the original JUnit test suites for the corresponding program version. Since one of the goals of this study is to understand the impact of test granularity on the effectiveness of TCP techniques, we introduce two groups of experiments in our empirical study based on two test-case granularities: (i) the test-method and (ii) the test-class granularity. The numbers of test cases on

TABLE 2

The stats of the Subject Programs: Size: #Loc; TM: #Test Cases at Method Level; TC: #test Cases at Class Level; All: #all Mutation Faults; Detected: #Faults Can Be Detected by Test Cases

Subject Programs	Size	Tests		Mutants	
		#TM	#TC	Detected	All
P1-geojson-jackson	1,151	44	13	301	717
P2-statsd-jvm-profiler	1,355	29	12	290	708
P3-stateless4j	1,756	61	10	392	696
P4-jarchivelib	1,940	22	12	655	948
P5-JSONassert	1,957	121	10	935	1,116
P6-java-faker	2,069	28	11	392	600
P7-jackson-datatype-joda	2,409	57	8	675	1,212
P8-Java-apns	3,234	87	15	412	1,122
P9-pusher-websocket-java	3,259	199	11	851	1,470
P10-gson-fire	3,421	55	14	847	1,064
P11-jackson-datatype-guava	3,994	91	15	313	1,832
P12-dictomaton	4,099	53	11	2,024	10,857
P13-jackson-uuid-generator	4,158	45	6	802	2,039
P14-JAdventure	4,416	35	10	738	5,098
P15-exp4j	4,617	285	9	1,365	1,563
P16-jumbl	4,623	103	15	610	1,192
P17-efflux	4,940	41	10	1,190	2,840
P18-metrics-core	5,027	144	28	1,656	5,265
P19-low-gc-memuffers	5,198	51	18	1,861	3,654
P20-xembly	5,319	58	16	1,190	2,546
P21-scribe-java	5,355	99	18	563	1,622
P22-jpush-api-java-client	5,462	65	10	822	2,961
P23-gdx-artemis	6,043	31	20	968	1,687
P24-protoparser	6,074	171	14	3,346	4,640
P25-commons-cli	6,601	317	26	2,362	2,801
P26-mp3agic	6,939	205	19	3,362	6,391
P27-webbit	7,363	131	25	1,268	3,833
P28-RestFixture	7,421	268	30	2,234	3,278
P29-LastCalc	7,707	34	13	2,814	6,635
P30-jackson-dataformat-csv	7,850	98	27	1,693	6,795
P31-skype-java-api	8,264	24	16	885	6,494
P32-lambda	8,510	252	35	3,382	4,341
P33-jackson-dataformat-xml	8,648	134	45	1,706	4,149
P34-jopt-simple	8,778	511	79	2,325	2,525
P35-jline2	8,783	130	16	3,523	8,368
P36-javapoet	9,007	246	16	3,400	4,601
P37-Liqa	9,139	235	58	7,962	18,608
P38-cassandra-reaper	9,896	40	12	1,186	5,105
P39-JSqlParser	10,335	313	19	15,698	32,785
P40-raml-java-parser	11,126	190	36	4,678	6,431
P41-redline-smalltalk	11,228	37	9	1,834	10,763
P42-user-agent-utils	11,456	62	7	376	688
P43-javaewah	13,293	229	11	6,307	11,939
P44-jsoup-learning	13,505	380	25	7,761	13,230
P45-wsc	13,652	16	8	1,687	17,942
P46-rome	13,874	443	45	4,920	10,744
P47-JActor	14,171	54	43	132	1,375
P48-RoaringBitmap	16,341	286	15	9,709	13,574
P49-JavaFastPFOR	17,695	42	8	46,429	64,372
P50-jprotobuf	21,161	48	18	1,539	10,338
P51-worldguard	24,457	148	12	1,127	25,940
P52-commons-jxpath	24,910	411	39	13,611	24,369
P53-commons-io	27,263	1125	92	7,630	10,365
P54-nodebox	32,244	293	40	7,824	36,793
P55-asterisk-java	39,542	220	39	3,299	17,664
P56-ews-java-api	46,863	130	28	2,419	31,569
P57-commons-lang	61,518	2388	114	25,775	32,291
P58-joda-time	82,998	4,026	122	20,957	28,382
Total	714,414	15,441	1,463	245,012	542,927

TABLE 3
Mutation Operators Used

ID	Mutation Operator
M0	Conditional Boundary Mutator
M1	Constructor Call Mutator
M2	Increments Mutator
M3	Inline Constant Mutator
M4	Invert Negs Mutator
M5	Math Mutator
M6	Negate Conditionals Mutator
M7	Non-Void Method Call Mutator
M8	Remove Conditional Mutator
M9	Return Vals Mutator
M10	Void Method Call Mutator
M11	Remove Increments Mutator
M12	Member Variable Mutator
M13	Switch Mutator
M14	Argument Propagation Mutator

test-method level and test-class level are shown in Columns 3 & 4 of Table 2 respectively.

One goal of this empirical study is to compare the effectiveness of different test prioritization techniques by evaluating their fault detection capabilities. Thus, each technique will be evaluated on a set of program faults introduced using mutation analysis. As mutation analysis has been widely used in regression test prioritization evaluations [13], [46], [73], [75] and has been shown to be suitable in simulating real program faults [4], [33], this is a sensible method of introducing program defects. We applied all the 15 available mutation operators from the PIT [54] mutation tool (Version 1.1.7) to generate mutants for each project. All mutation operators are listed in Table 3 and their detailed definitions can be found on the PIT website [52] and on our online appendix [48], available in the online supplemental material. We utilized PIT to determine the set of faults that can be detected by the test suites for each of our subject programs. When running the subject program's JUnit test suite via the PIT Maven plugin, test cases are automatically executed against each mutant, PIT records the corresponding test cases capable of killing each mutant. By analyzing the PIT reports, we obtained the information (e.g., fault locations) for each mutant and all the test cases that can detect it. Note that the typical implementation of PIT stops executing any remaining tests against a mutant once the mutant is killed by some earlier test to save time. However, for the purpose of obtaining a set of "killable" mutants, this is undesirable. Thus, we modified PIT to force it to execute the remaining tests against a mutant even when the mutant has been killed. Since not all produced mutants can be detected/covered by test cases, only mutants that can be detected by at least one test case are included in our study. The numbers of detected mutants and the numbers of all mutants are shown in Columns 5 and 6 of Table 2 respectively. As the table shows, the numbers of detected mutants range from 132 to 46,429. There are of course certain threats to validity introduced by such an analysis, namely the the potential bias introduced by the presence of equivalent and trivial mutants [2], [5]. We summarize the steps we take in our methodology to mitigate this threat in Section 5.

TABLE 4
Studied TCP Techniques

Type	Tag	Description
Static	TP_{cg-tot}	Call-graph-based (total strategy)
	TP_{cg-add}	Call-graph-based (additional strategy)
	TP_{str}	The string-distance-based
	$TP_{topic-r}$	Topic-model-based using R-Ilda package
	$TP_{topic-m}$	Topic-model-based using Mallet
Dynamic	TP_{total}	Greedy total (statement-level)
	TP_{add}	Greedy additional (statement-level)
	TP_{art}	Adaptive random (statement-level)
	TP_{search}	Search-based (statement-level)

3.3 Design of the Empirical Study

As discussed previously (Section 2), we limit the focus of this study to TCP techniques that do not require additional inputs, such as code changes or software requirements that may require extra effort or time to collect or may be unavailable. We select two white-box and two black-box static techniques, and four white-box dynamic techniques with statement-level coverage as the subject techniques for this study, which are listed in Table 4. We sample from both white and black box approaches as the major goal of this study is to examine the effectiveness and trade-offs of static and dynamic TCPs under the assumption that both the source code of the subject application, as well as the test cases are available. It is worth noting that our evaluation employs *two* versions of the static topic model-based technique, as when contacting the authors of [63], they suggested that an implementation using the Mallet [49] tool would yield better results than their initial implementation in R [63]. There are various potential coverage granularities for dynamic techniques, such as statement-level, method-level and class-level. Previous research showed that statement-level TCP techniques perform the best [27], [50]. Thus, in our study, we choose statement-level coverage for the dynamic TCP techniques. We now describe the experimental procedure utilized to answer each RQ posed above.

RQ₁. The goal of RQ₁ is to compare the effectiveness of different TCP techniques, by evaluating their fault detection capabilities. Following existing work [46], [73], we fixed the number of faults for each subject program. That is, we randomly chose 500 different mutation faults and partitioned the set of all faults into groups of five (e.g., a mutant group) to simulate each faulty program version. Thus, 100 different faulty versions (i.e., $500/5 = 100$) were generated for each program. If a program has less than 500 mutation faults, we use all detected mutation faults for this program and separate these faults into different groups (five faults per group). For the static techniques, we simply applied the techniques as described in Sections 2 & 3.4 to the test and source code of each program to obtain the list of prioritized test cases for each mutant group. For the dynamic techniques, we obtained the coverage information of the test-cases for each program. We then used this coverage information to implement the dynamic approaches as described in Sections 2 & 3.4. Then we are able to collect the fault detection information for each program according to the fault locations.

To measure the effectiveness in terms of rate of fault detection for each studied test prioritization technique, we utilize the well-accepted Average Percentage of Faults Detected (APFD) metric in TCP domain [15], [17], [18], [21], [56], [57], [76]. Recall that every subject program has 100 mutant groups (five mutations per group). Thus, we created 100 faulty versions for each subject (each version contains five mutations) and ran all studied techniques over these 100 faulty versions. That is, running each technique 100 times for each subject. Then, we performed statistical analysis based on the APFD results of these 100 versions. To test whether there is a statistically significant difference between the effectiveness of different techniques, we first performed an one-way ANOVA analysis on the mean APFD values for all subjects and a Tukey HSD test [62], following the evaluation procedures utilized in related work [46], [50]. The ANOVA test illustrates whether there is a statistically significant variance between all studied techniques and the Tukey HSD test further distinguishes techniques that are significantly different from each other, as it classifies them into different groups based on their mean APFD values [62]. These statistical tests give a statistically relevant overview of whether the mean APFD values for the subject programs differ significantly. Additionally, we performed a Wilcoxon signed-rank test between each pair of TCP techniques for their average APFD value across all subject techniques, to further illustrate the relationship between individual subject programs. We choose to include this non-parametric test since we cannot make assumptions about whether or not the data under consideration is normally distributed.

RQ₂. Although APFD has been widely used for TCP evaluation, it assumes that each test takes the same amount of time, which may not be always accurate in practice. The goal of this RQ is to examine the effectiveness of TCP techniques in terms of the APFDc metric, which considers both the execution time and severities of detected faults. We also compare the results of the APFDc with those of the APFD for understanding the performance of different types of metrics in the TCP area. However, there is no clearly-defined way to estimate the severities for the detected faults, and no widely-used tool to collect this information, making it hard to measure fault severity. Therefore, following previous work [24], we consider all faults to share the same severity level. Thus, in the context of our empirical study, APFDc reduces to the following equation:

$$APFDc = \frac{\sum_{i=1}^m \sum_{j=TF_i}^n t_j - \frac{1}{2} t_{TF_i}}{\sum_{i=1}^n t_i * m}, \quad (3)$$

where n is the number of test cases in T , m is the number of faults, TF_i is the position of the first test case in T' that detects fault, and $i, t_1, t_2, \dots, t_n$ are the execution costs for all the n test cases. To measure test execution costs, we use the Maven Surefire Plugin to trace the start and end events of each test to record the corresponding execution time. Similar as RQ₁, we performed both of an one-way ANOVA analysis on the mean APFDc values for all subjects and a Tukey HSD test to further understand the whether there is a statistically significant variance between the performance of

the studied techniques in terms of APFDc values. In addition, we further examined the relationship between the two metrics, APFD and APFDc, to understand the differences in effectiveness of TCP techniques. We utilize the Kendall rank correlation coefficient τ [59] to compare the results of these two metrics. Kendall rank correlation coefficient τ is commonly used to examine the relationship between two ordering quantities (i.e., observations of two variables). The coefficient ranges in value from -1 to 1 , with values closer to 1 indicating similarity and values closer to -1 indicating dissimilarity. When the value is close to 0 , these two quantities are considered independent. For example, in the context of our study, we have two quantities, APFD and APFDc values. Thus, in the context of our study, if the values of APFD values across all TCP techniques are similar to APFDc values, the Kendall tau rank coefficient τ would be closer to 1 . Otherwise, it would be closer to -1 . Since there is no guarantee that the relationship between APFD and APFDc values are linear, we chose Kendall τ_b coefficient in our study, following prior work [26]:

$$\tau_b = \frac{n_c - n_d}{\sqrt{(n_0 - n_1)(n_0 - n_2)}}$$

$n_c = \#$ of concordant pairs
 $n_d = \#$ of discordant pairs
 $n_0 = n(n-1)/2$
 $n_1 = \sum_i t_i(t_i-1)/2$
 $n_2 = \sum_j u_j(u_j-1)/2$
 $t_i = \#$ of tied values in i th tie group for 1st quantity
 $u_j = \#$ of tied values in j th tie group for 2nd quantity,

(4)

RQ₃. The *goal* of this RQ is to analyze the impact of different test granularities on the effectiveness of TCP techniques. Thus, we choose two granularities: test-method and test-class levels. The test-method level treats each JUnit test method as a test case, while test-class level treats each JUnit test class as a test case. We examine both the effectiveness and similarity of detected faults for both granularities.

RQ₄. The *goal* of this RQ is to investigate the impact of different program sizes on the effectiveness of TCP techniques. Thus, we measure the size for each subject program in terms of its Lines of Code (LOC). To examine whether TCP techniques tend to perform differently on programs of different sizes we classify the programs into two groups, a set of *smaller* programs and a set of *larger* programs. These two groups were created by ordering our subject programs in increasing order of LOC and splitting the ordered list in the middle. This results in two groups of 29 subject programs, the first group containing smaller programs and the second group containing larger programs.

RQ₅. The *goal* of this RQ is to understand the effectiveness of TCP techniques in a software evolution scenario. To accomplish this we apply different TCP techniques across different versions of each subject program. More specifically, tests are prioritized using the information from a given previous program version, and the prioritized set of test cases is then applied to faulty variants of the most

recent program version. The faulty variants are created using the same methodology described for RQ₁. This methodology closely follows that of previous work [46] and allows us to investigate whether the performance of TCP techniques remains stable, decreases, or increases as software evolves. In our study, we collect different versions for each subject program exactly following the methodology proposed in [46]. For each subject, we start from the most current version and collect one version per ten commits moving backward through the commit history. We then discard those programs that did not successfully compile and those that are not applicable to our tools. Due to the extremely large volume of data and the time cost of running these experiments, we randomly chose 12 subject programs to investigate this research question. Note that the numbers of versions (i.e., 66) and subject programs (i.e., 12) used in this work are larger than all prior TCP work considering evolutionary scenarios (e.g., the recent work by Lu et. al. [46] used 53 versions of 8 real-world programs).

RQ₆. The *goal* of this RQ is to examine the impact of mutant quantity on the effectiveness for TCP techniques in terms of APFD and APFDc values. In our default experimental setting, we have 100 groups of mutation faults, and each group contains five mutants following prior work [27], [46], [50], [73]. However, in practice, the number of faults within a buggy version can be more than or less than five. Therefore, to better understand the impact of fault quantity per group, we generate different number of faults (i.e., 1 to 10) within each of the 100 constructed fault groups for each subject program. Note that we may have less than 100 fault groups when the number of mutants are small for some subjects. That is, we repeat all our prior experiments 10 times, each time recording the APFD and APFDc values for all studied techniques under 100 fault groups with a different number of faults (from 1 to 10). Finally, we perform Kendall rank τ_b coefficient analysis to understand the relationship between the results for the mutation groups with different sizes and the results of the default setting (i.e., with 5 faults within each group). That is, we perform Kendall analysis to compare each fault-quantity setting (i.e., 100 groups mutation faults and the size of each fault group varies from 1 to 10) to the mutation faults with the default setting. Intuitively, if the values of Kendall τ_b coefficient are close to 1 , the TCP techniques perform similarly between fault groups of varying sizes and fault groups with the default size, implying that the quantity of mutation faults does not impact TCP evaluation.

RQ₇. The *goal* of this RQ is to understand the impact of the mutant types (i.e., those mutants generated with different operators) on the effectiveness for TCP techniques in terms of APFD and APFDc values. Intuitively, we first classified mutants into different groups based on their corresponding operators. That is, the mutation faults generated by the same operators would be classified into the same group. In our empirical study, we utilized all 15 built-in mutation operators in PIT. Thus, we have 15 types of mutation faults for each subject program. We evaluate TCP techniques across these 15 types of mutation faults with the default setting, where for each operator we randomly choose 500 mutants and separate them into 100 groups (each group contains 5 mutation faults). Note that we may have less than 100 fault groups when the number of mutants are small for some

mutant types. Then, TCP techniques are evaluated based on these groups of mutation faults. Finally, we compare the results for different types of mutation faults with our default fault seeding (i.e., randomly including different types of faults) under the same default setting (i.e., 100 mutated groups and each group contains 5 mutation faults). Similar as RQ_4 , we chose Kendall rank tau coefficient to measure the relationship between them to check if the type of mutation fault impacts TCP evaluation.

RQ_8 . The goal of this RQ is to analyze the similarity of detected faults for different techniques to better understand the level of equivalency of differing strategies. It is clear that this type of analysis is important, as while popular metrics such as APFD measure the *effectiveness* between two different techniques, this does not reveal the similarity of the test cases in terms of uncovered faults. For instance, let us consider two TCP techniques A and B. If technique A achieves an APFD of ≈ 60 percent and technique B achieves an APFD of ≈ 20 percent, while this gives a measure of relative effectiveness, the APFD does not reveal how similar or orthogonal the techniques are in terms of the faults detected. For instance, all of the faults uncovered by top ten test cases from technique B could be different than those discovered by top ten test cases from technique A, suggesting that the techniques may be *complimentary*. To evaluate the similarity between different TCP techniques, we utilize and build upon similarity analysis used in recent work [28], [29] and construct binary vector representations of detected faults for each technique and then calculate the distance between these vectors as a similarity measure.

We employ two methodologies in order to give a comprehensive view of the similarity of the studied TCPs. At the core of both of these techniques is a measure of similarity using the Jaccard distance to determine the distance between vectorized binary representations of detected faults (where a 1 signifies a found fault and a 0 signifies an undiscovered fault) for different techniques across individual or groups of subject programs. We use the following definition [28]:

$$J(T_A^i, T_B^i) = \frac{|T_A^i \cap T_B^i|}{|T_A^i \cup T_B^i|}, \quad (5)$$

where T_A^i represents the binary vectorized discovered faults of some studied technique A after the execution of the i th test case in the techniques prioritized set, and T_B^i represents the same meaning for some studied technique B and $0 \leq J(T_A^i, T_B^i) \leq 1$. While we use the same similarity metric as in [28], we report two types of results: 1) results comparing the similarity of the studied static and dynamic techniques using the average Jaccard coefficient across all subjects at different test-case granularities, and 2) results comparing each technique in a pair-wise manner for each subject program. For the second type of analysis, we examine each possible pair of techniques and rank each subject program according to Jaccard coefficient as highly similar (1.0 - 0.75), similar (0.749 - 0.5), dissimilar (0.49 - 0.25), or highly dissimilar (0.249-0). This gives a more informative view of how similar two techniques might be for different subject programs. To construct both types of binary fault vectors, we use the same fault selection methodology used to calculate the APFD, that is, we randomly sample 500 faults from the set of discoverable faults for each subject.

In addition, we also want to understand whether the studied TCP techniques' most highly prioritized test cases uncover comparatively different numbers of mutants generated by different operators. Thus, for different cut points, particularly the top cut points (e.g., 10 percent), we examine the both the total number and relative percentages of different types of mutants detected by each TCP technique to better understand the types of mutants which are easily detected by most highly prioritized test cases for different techniques.

RQ_9 . The final goal of our study is to understand the efficiency of static techniques, in terms of execution costs. Note that, we only focus on the efficiency of static techniques, since dynamic techniques are typically run on the previous version of a program to collect coverage information, and thus the temporal overhead is quite high and well-studied. To evaluate the efficiency of static techniques, we collect two types of time information: the time for pre-processing and the time for prioritization. The time for pre-processing contains different phases for different techniques. For example, TP_{cg-tot} and TP_{cg-add} need to build the call graphs for each test case. TP_{str} needs to analyze the source code to extract identifiers and comments for each test case. Besides, TP_{topic} needs to pre-process extracted textual information and use the R-LDA package and Mallet [49] to build topic models. The time for prioritization refers to the time cost for TCP on different subjects.

3.4 Tools and Experimental Hardware

We reimplemented all of the studied dynamic and static TCPs in Java according to the specifications and descriptions in their corresponding papers since the implementations were not available from the original authors and had to be adapted to our subjects. Three of the authors carefully reviewed and tested the code to make sure the reimplementation is reliable.

TP_{cg-tot}/TP_{cg-add} . Following the paper by Zhang et al. [76], we use the IBM T. J. Watson Libraries for Analysis (WALA) [66] to collect the RTA static call graph for each test, and traverse the call graphs to obtain a set of relevant methods for each test case. Then, we implement two greedy strategies (i.e., total and additional) to prioritize test cases.

TP_{str} . Based on the paper by Ledru et al. [43], each test case is treated as one string without any preprocessing. Thus, we directly use JDT [31] to collect the textual test information for each JUnit test, and then calculate the Manhattan distances between test cases to select the one that is farthest from the prioritized test cases.

$TP_{topic-r}$ and $TP_{topic-m}$. Following the topic-based TCP paper [63], we first use JDT to extract identifiers and comments from each JUnit test, and then pre-process those (e.g., splitting, removing stop words, and stemming). To build topic models, we used the R-LDA package [42] for $TP_{topic-r}$ and Mallet [49] for $TP_{topic-m}$. All parameters are set with previously used values [9], [63]. Finally, we calculated the Manhattan distances between test cases, and selected the ones that are farthest from the prioritized test cases.

Dynamic TCP Techniques. We use the ASM bytecode manipulation and analysis toolset [7] to collect the coverage information for each test. Specifically, in our empirical study, it obtains a set of statements that can be executed by each test method or test class. The greedy techniques are

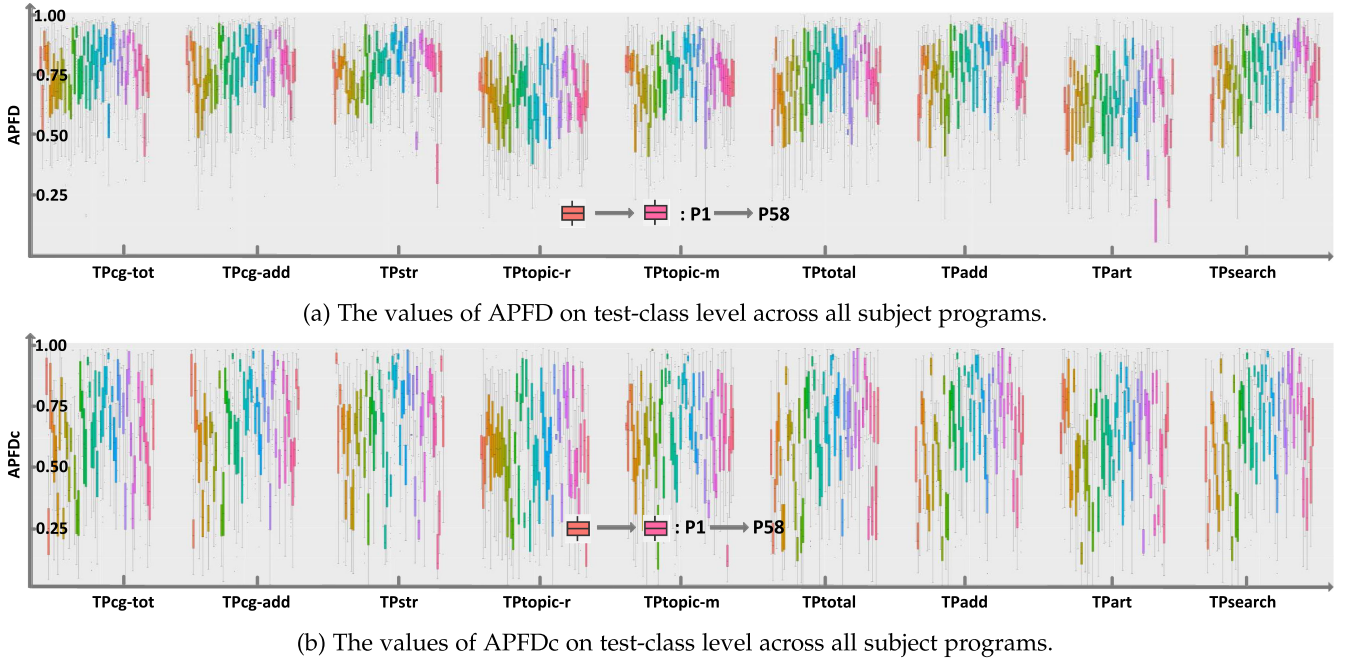


Fig. 1. The box-and-whisker plots represent the values of APFD and APFDc for different TCP techniques at test-class level. The x -axis represents the APFD and APFDc values. The y -axis represents the different techniques. The central box of each plot represents the values from the lower to upper quartile (i.e., 25 to 75 percentile).

replicated based on the paper by Rothermel et al. [56]. For the ART and search-based techniques, we follow the methodology described in their respective papers [32], [45].

Experimental Hardware. The experiments were carried out on Thinkpad X1 laptop with Intel Core i5-4200 2.30 GHz processor and 8 GB DDR3 RAM and eight servers with 16, 3.3 GHz Intel(R) Xeon(R) E5-4627 CPUs, and 512 GB RAM, and one server with eight Intel X5672 CPUs and 192 GB RAM. All the execution time information (i.e., both of the execution time to run TCP techniques and the execution time for each test case) was collected on the laptop to ensure that the analysis for time costs is consistent.

4 RESULTS

In this section, we outline the experimental results to answer the RQs listed in Section 3.

4.1 RQ₁ & RQ₂ & RQ₃: Effectiveness of Studied Techniques Measured by APFD and APFDc at Different Granularities

4.1.1 Results at Test Class Level

The values of APFD across all subjects at the test class level are shown in Fig. 1a and Table 5. Based on the results, we

observe that, somewhat surprisingly at the test-class level, the static TP_{cg-add} technique performs the best across all studied TCP techniques (including all dynamic techniques) with an average APFD value of 0.790 (see Table 5). Among the static techniques, TP_{cg-add} performs best, followed by TP_{cg-tot} , TP_{str} , $TP_{topic-m}$ and $TP_{topic-r}$. The best performing dynamic technique at class-level is TP_{add} followed by TP_{search} , TP_{total} , and TP_{art} . It is notable that at test-class level granularity, the most effective static technique TP_{cg-add} performs even better than the most effective dynamic technique TP_{add} in terms of APFD, i.e., 0.790 versus 0.769. The experimental results on APFDc values further confirm the above finding. Shown in Fig. 1b and Table 5, the static TP_{cg-add} technique outperforms all the studied TCP techniques with an average APFDc value of 0.679, whereas even the most effective dynamic TP_{add} only achieves an average APFDc value of 0.650. Furthermore, the Kendall τ_b Rank Correlation value of 0.722 also demonstrates that APFDc values are generally consistent with APFD values at the test class level. Therefore, at the test-class level, the call-graph based strategies can even outperform dynamic-coverage based strategies, which is notable. Additionally, overall the static techniques outperform the dynamic techniques at the test-class level. One potential reason for this is that many

TABLE 5
Results for the ANOVA and Tukey HSD Tests on the Average APFD and APFDc Values at Test-Class Level, Which Are Depicted in Fig. 1

Metrics	Analysis	TP_{cg-tot}	TP_{cg-add}	TP_{str}	$TP_{topic-r}$	$TP_{topic-m}$	TP_{total}	TP_{add}	TP_{art}	TP_{search}	p-value	τ_b
APFD	Avg HSD	0.778 A	0.790 A	0.777 A	0.675 B	0.745 A	0.738 A	0.769 A	0.633 B	0.765 A	1.777e-18	0.722
APFDc	Avg HSD	0.652 A	0.679 A	0.667 A	0.574 A	0.657 A	0.614 A	0.650 A	0.612 A	0.649 A	0.154	

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFDc and average APFD.

Authorized licensed use limited to: AIT Austrian Institute of Technology. Downloaded on August 08, 2026 at 18:53:33 UTC from IEEE Xplore. Restrictions apply.

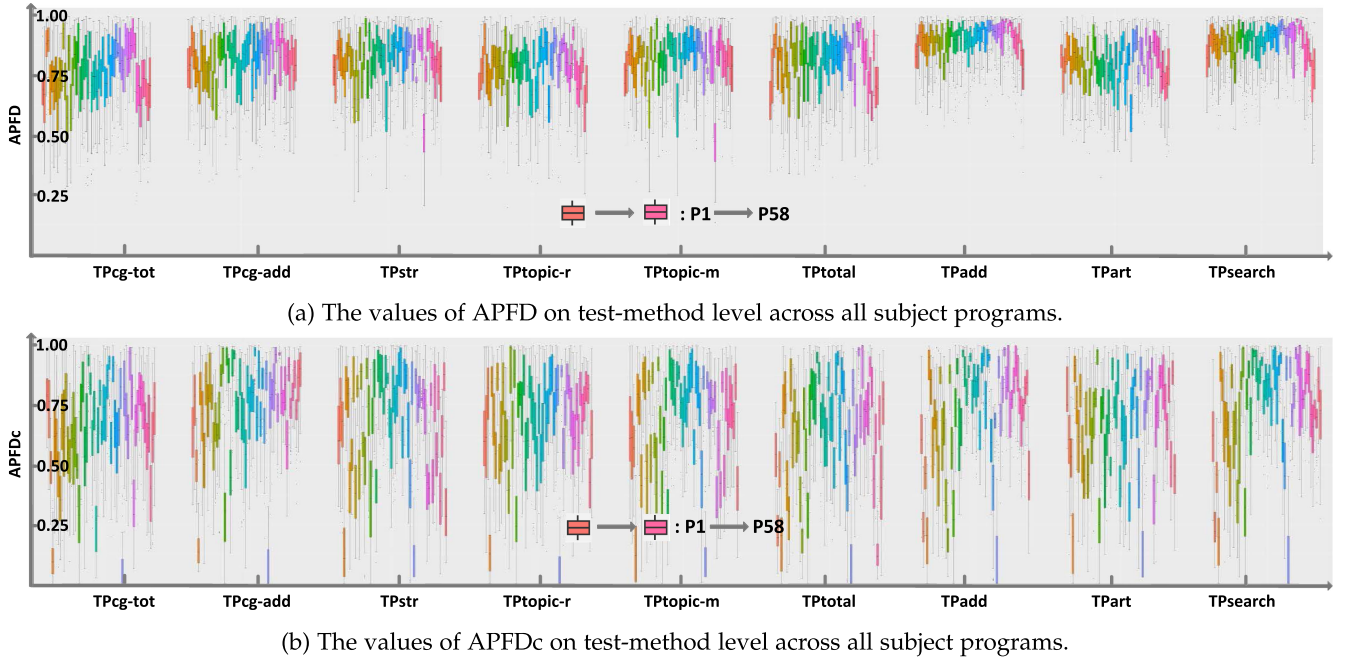


Fig. 2. The box-and-whisker plots represent the values of APFD and APFDc for different TCP techniques at test-method level. The x -axis represents the APFD and APFDc values. The y -axis represents the different techniques. The central box of each plot represents the values from the lower to upper quartile (i.e., 25 to 75 percentile).

program statements are covered several times by tests at the test-class level, making the traditional dynamic techniques less precise, since they do not consider the number of times that a statement is covered.

While Fig. 1 shows the detailed APFD and APFDc values for each studied subject at test-class level, Fig. 3a further shows the ranges of APFD and APFDc values across all subjects at test-class level, reflecting the robustness of the studied approaches across both metrics. For APFD, the range of average values across all subjects at test-class level for TP_{add} is the smallest (i.e., 0.523-0.947), implying that the performance of TP_{add} is usually stable despite differing subjects for this metric. Conversely, the ranges of APFD values for TP_{str} and TP_{art} are much larger (0.391-0.917 for TP_{str} , 0.187-0.852 for TP_{art}), implying that their performance varies across different types of subjects. However, we observe different trends for the APFDc metric. The ranges of APFDc values are all much larger than those of APFD values. This is most likely due to the fact that APFDc considers execution times, which we found to be randomly distributed, resulting in a larger variation in results across different subjects.

To further investigate the finding that static techniques tend to have a higher variance in terms of effectiveness depending on the program type, we investigated further by

inspecting several subject programs. One illustrative example is that *scribe-java* scores 0.646 and 0.606 for the average values of APFD under TP_{str} and $TP_{topic-r}$ respectively, which are notably worse than the results of TP_{cg-tot} (0.718) and TP_{cg-add} (0.733). To understand the reason for this discrepancy, we analyzed the test code and found that *Scribe-java* is documented/written more poorly than other programs. For instance, the program uses meaningless comments and variable names such as '*param1*', '*param2*', '*v1*', '*v2*' etc. This confirms the previously held notion [63] that static techniques which aim to prioritize test-cases through text-based diversity metrics experience performance degradation when applied to test cases written in a poor/generic fashion. It also suggests that researchers may take the subject characteristics into account when choosing TCP techniques in future work.

4.1.2 Results at Test Method Level

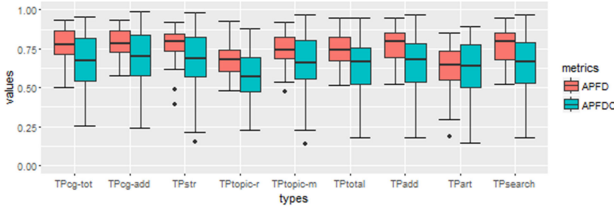
To further answer RQ_3 we ran all of the subject TCP techniques on the subject programs at the test-method level so that we can compare to the results at the test-class level outlined above (see Section 4.1.1). The results are shown in Fig. 2 and Table 6. In terms of APFD, when examining the static techniques with the *test-method granularity*, they perform differently as compared to the results on the test-class

TABLE 6
Results for the ANOVA, and Tukey HSD Tests on the Average APFD and APFDc Values at Test-Method Level, Which Are Depicted in Fig. 2

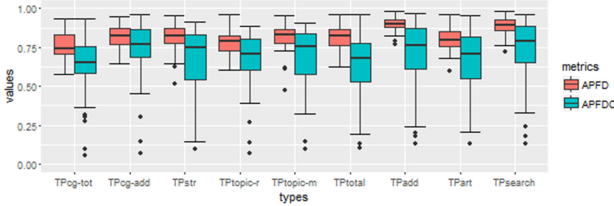
Metrics	Analysis	TP_{cg-tot}	TP_{cg-add}	TP_{str}	$TP_{topic-r}$	$TP_{topic-m}$	TP_{total}	TP_{add}	TP_{art}	TP_{search}	p-value	τ_b
APFD	Avg HSD	0.764 C	0.818 B	0.813 B	0.781 BC	0.817 B	0.809 B	0.898 A	0.798 BC	0.885 A	2.568e-28	0.556
APFDc	Avg HSD	0.638 A	0.737 A	0.671 A	0.678 A	0.679 A	0.633 A	0.708 A	0.669 A	0.735 A	0.053	

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFDc and average APFD.

Authorized licensed use limited to: AIT Austrian Institute of Technology. Downloaded on August 08, 2026 at 18:53:33 UTC from IEEE Xplore. Restrictions apply.



(a) The values of APFD and APFDc for different TCP techniques across all subject programs on test-class level.



(b) The values of APFD and APFDc for different TCP techniques across all subject programs on test-method level.

Fig. 3. The box-and-whisker plots represent the values of APFDc for different TCP techniques at different test granularities. The x -axis represents the APFDc values. The y -axis represents the different techniques. The central box of each plot represents the values from the lower to upper quartile (i.e., 25 to 75 percentile).

level. For example, although TP_{cg-add} still performs the best among static techniques, it is inferior to the most effective dynamic technique TP_{add} (0.818 versus 0.898). This finding is consistent with previous studies [27]. Also, surprisingly, $TP_{topic-m}$ (0.817) achieves almost the same average APFD values as TP_{cg-add} , followed by TP_{str} , $TP_{topic-r}$ and TP_{cg-tot} respectively. It is worth noting that the effectiveness of the topic-model based technique varies quite dramatically depending on the tools used for its implementation: Mallet [49] significantly outperforms the R-based implementation. Also, there is less variation in the APFD values at the test-method level compared to those at the test-class level, as shown in Figs. 2 and 3.

In terms of APFDc, the results for test-method level are generally consistent with the results on test-class level. For example, while TP_{search} tend to be the most effective dynamic technique, the static TP_{cg-add} outperforms all the studied static and dynamic techniques. The likely reason is that dynamic techniques tend to favor tests with higher coverage, which tend to cost more time to execute, leading to limited effectiveness in actual time cost reduction. The results of the HSD analysis on the APFDc values at the test-method level, indicate that all techniques are grouped into the same level (level A), implying that different TCP techniques share similar performance based on APFDc values, which is also consistent with the results of APFDc values at the test-class level. When examining the ranges of APFDc values for the test-method level (see Figs. 2 and 3), we find the APFDc values vary dramatically between subject programs. When comparing the results of APFD and APFDc values at the test-method level, the Kendall τ_b rank coefficient τ_b is 0.556, implying that the APFDc results are less consistent with the APFD results at the test-method level. The reason is likely that test execution time distributions which are uncontrolled have large impacts the more effective/stable test-method-level results.

In addition, as a whole, the effectiveness of the dynamic techniques outpaces that of the static techniques

at method-level granularity for the APFD metric, with TP_{add} performing the best of all studied techniques (0.898). For the cost-cognizant APFDc metric, although there are no clear trends, the static techniques tend to perform even better than dynamic techniques, indicating the limitations of dynamic information for actual regression testing time reduction. Overall, on average, almost all static and dynamic TCPs perform better on the test-method level as compared to the results on the test-class level in terms of both APFD and APFDc. Logically, this is not surprising, as using a finer level of granularity (e.g., prioritizing individual test-methods) gives each technique more flexibility, which leads to more accurate targeting and prioritization.

Finally, to check for statistically significant variations in the mean APFD and APFDc values across all subjects and confirm/deny our null hypothesis for RQ_1 and RQ_2 , we examine the results of the one-way ANOVA and Tukey HSD tests. The ANOVA test for APFD values, given in the second to last column of Tables 5 and 6, are well below our established significance threshold of 0.05, thus signifying that the subject programs are statistically different from one another. This rejects the null hypothesis H_0 and we conclude that there are statistically significant differences between different TCP techniques in terms of APFD. The results of the Tukey HSD test also illustrate the statistically significant differences between the static and dynamic techniques, by grouping the techniques into categories with *A* representing the best performance and the following letters (e.g., *B*) representing groups with worse performance. We see that the groupings are similar for static and dynamic techniques. In order to illustrate the individual relationships between strategies, we present the results of the Wilcoxon signed rank test for all pairs of techniques at both granularity levels in Tables 7 and 8. The shaded cells represent statistically significant differences between techniques across all the subjects (e.g., $p < 0.05$). The Wilcoxon signed rank test further confirms that different techniques have statistically different APFD values at both test-class and test-method levels, as indicated by the shaded boxes. On the contrary, the results for APFDc ANOVA and HSD tests lead to different observations – different techniques *generally* do not have statistically different APFDc values (as shown in Tables 5 and 6), indicating that both static and dynamic techniques tend to perform similarly for APFDc values. The Wilcoxon signed rank test for APFDc values of all pairs of techniques is shown in Table 8. The small number of shaded cells (i.e., $p < 0.05$) further confirms that different techniques tend to perform equivalently for APFDc. The likely reason for this is that APFDc is impacted by an additional randomly-distributed factor, i.e., tests tend to have randomly distributed execution times, leading to the observed results. It should be noted that in contrast to our previous work [47], our results for the HSD show less variance between the different approaches for APFD at both test-class and test-method level. This means that the approaches were grouped in fewer differing groups by the HSD test, indicating performance that is more comparatively similar. This illustrates the affect of generalizing across more subject programs.

TABLE 7

The Table Shows the Results of Wilcoxon Signed Rank Test on the Average APFD values for Each Pair of TCP Techniques

	T2		T3		T4		T5		T6		T7		T8		T9	
T1	0.02	< 0.01	> 0.05	< 0.01	< 0.01	> 0.05	< 0.01	< 0.01	0.02	< 0.01	> 0.05	< 0.01	< 0.01	< 0.01	> 0.05	< 0.01
T2	-	-	> 0.05	> 0.05	< 0.01	< 0.01	< 0.01	> 0.05	< 0.01	> 0.05	> 0.05	< 0.01	< 0.01	> 0.05	> 0.05	< 0.01
T3	-	-	-	-	< 0.01	< 0.01	< 0.01	> 0.05	< 0.01	> 0.05	> 0.05	< 0.01	< 0.01	> 0.05	> 0.05	< 0.01
T4	-	-	-	-	-	-	< 0.01	< 0.01	< 0.01	< 0.01	< 0.01	< 0.01	0.05	0.02	< 0.01	< 0.01
T5	-	-	-	-	-	-	-	-	> 0.05	> 0.05	0.03	< 0.01	< 0.01	0.04	0.05	< 0.01
T6	-	-	-	-	-	-	-	-	-	-	< 0.01	< 0.01	< 0.01	> 0.05	< 0.01	< 0.01
T7	-	-	-	-	-	-	-	-	-	-	-	< 0.01	< 0.01	< 0.01	0.04	< 0.01
T8	-	-	-	-	-	-	-	-	-	-	-	-	-	-	< 0.01	< 0.01

The techniques T1 to T9 refer to TP_{cg-tot} , TP_{cg-add} , TP_{str} , $TP_{topic-r}$, $TP_{topic-m}$, TP_{total} , TP_{add} , TP_{art} , TP_{search} respectively. For each pair of TCP techniques, there are two sub-cells. The first one refers to the p -value at test-class level and the second one refers to the p -value at test-method level. The p -values are classified into three categories, 1) $p > 0.05$, 2) $0.01 < p < 0.05$, 3) $p < 0.01$. The p -values for categories $p > 0.05$ and $p < 0.01$ are presented as $p > 0.05$ and $p < 0.01$ respectively. If a p -value is less than 0.05, the corresponding cell is shaded.

In summary we answer RQ_1 , RQ_2 & RQ_3 as follows:

RQ₁: There is a statistically significant difference between the APFD values of the two types (e.g., static and dynamic) of studied techniques. On average, static technique TP_{cg-add} is the most effective technique at test-class level, whereas dynamic technique TP_{add} is the most effective technique at test-method level. Overall, the static techniques outperform the dynamic ones at test-class level, but the dynamic techniques outperform the static ones at test-method level.

RQ₂: For the APFDc values, there is no statistically significant difference between the studied static and dynamic techniques. APFDc values are generally consistent with APFD values at test-class level but relatively less consistent at test-method level. Similar to the results from RQ_1 , on average, static TP_{cg-add} technique is the most effective technique at the test-class level, and the static techniques outperform the dynamic ones as a whole at test-class level. However, at test-method level, TP_{cg-add} also performs best overall at the test-method level, indicating the superiority of static techniques to dynamic techniques in actual regression testing time reduction. Additionally, APFDc values vary more dramatically across all subject programs as compared to APFD values.

RQ₃: The test granularity significantly impacts the effectiveness of TCP techniques in terms of both APFD and APFDc, although the APFDc metric is affected to a much lesser extent. All the studied techniques perform better at test-method level as compared to test-class level. There is also less variation in the APFD values at method-level as compared to class-level, which signifies that the performance as measured by this metric is more stable at test-method level across the studied techniques.

4.2 Impact of Subject Program's Size

Since developers may apply TCP techniques to subject systems in various sizes in practice, it is important to understand the potential impact of program size on the performance of TCP techniques. Thus we examine the differences between the performance of our studied TCP techniques on our 29

smaller subject systems and 29 larger subject systems. Tables 9 and 10 present the TCP results at the test-class level on smaller and larger subject systems, respectively; similarly, Tables 11 and 12 present the TCP results at the test-method level on smaller and larger subject systems, respectively.

From the tables, we can make the following observations. First, TCP techniques tend to perform better on larger subject systems than smaller subject systems. For example, for both test-class and test-method level, all the studied TCP techniques perform better on larger subject systems in terms of both APFD and APFDc. One potential reason is that larger subject systems tend to have more tests, leaving enough room for TCP techniques to reach optimization thresholds. This finding also demonstrates the scalability of the studied TCP techniques. Second, at the test-class level, static TCP techniques tend to outperform dynamic TCP techniques in terms of both APFD and APFDc on both subsets of subject systems; in contrast, at the test-method level, static TCP techniques are inferior to dynamic TCP techniques on both subsets of subjects in terms of APFD, while TP_{cg-add} outperforms all the other studied dynamic and static TCP techniques on both subsets of subjects in terms of APFDc. This finding is consistent with our findings in RQ_1 and RQ_2 , indicating that subject size does not impact our findings when comparing the relative performance of the studied TCP techniques according to APFD and APFDc. Third, most studied TCP techniques perform better at test-method level as compared to test-class level in terms of both APFD and APFDc on both subsets of our subjects. This observation is also consistent with our comparative findings for RQ_3 .

RQ₄: All the studied TCP techniques tend to perform better on larger subject systems, indicating the scalability of the studied TCP techniques. However, when comparing the performance of different TCP techniques to each other on either the large or small programs, we find results consistent to using the entire set of programs (both in terms of APFD(c) and differing test-case granularities). Thus we can conclude that program size has little effect when comparing the relative performance of TCP techniques on a given subject.

4.3 Impact of Software Evolution

Figs. 4 and 5 present the impact of software evolution on the studied TCP techniques at the test-class and test-method

TABLE 8

The Table Shows the Results of Wilcoxon Signed Rank Test on the Average APFDc Values for Each Pair of TCP Techniques

	T2	T3	T4	T5	T6	T7	T8	T9
T1	0.02	< 0.01	0.03	> 0.05	0.04	> 0.05	> 0.05	> 0.05
T2	-	-	> 0.05	0.01	< 0.01	< 0.01	> 0.05	< 0.01
T3	-	-	-	-	0.02	> 0.05	> 0.05	> 0.05
T4	-	-	-	-	-	-	< 0.01	> 0.05
T5	-	-	-	-	-	-	-	> 0.05
T6	-	-	-	-	-	-	-	< 0.01
T7	-	-	-	-	-	-	-	< 0.01
T8	-	-	-	-	-	-	-	< 0.01

This table follows exactly the same format as Table 7.

TABLE 9

Results for the ANOVA and Tukey HSD Tests on the Average APFD and APFDc Values at Test-Class Level Across Smaller Subject Programs

Metrics	Analysis	TP _{cg-tot}	TP _{cg-add}	TP _{str}	TP _{topic-r}	TP _{topic-m}	TP _{total}	TP _{add}	TP _{art}	TP _{search}	p-value	τ_b
APFD	Avg	0.759	0.764	0.758	0.658	0.729	0.707	0.746	0.629	0.743	5.42E-8	0.5
	HSD	A	A	A	BC	AB	ABC	A	C	A		
APFDc	Avg	0.618	0.633	0.653	0.563	0.652	0.558	0.592	0.585	0.591	0.503	
	HSD	A	A	A	A	A	A	A	A	A		

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFDc and average APFD.

TABLE 10

Results for the ANOVA and Tukey HSD Tests on the Average APFD and APFDc Values at Test-Class Level Across Larger Subject Programs

Metrics	Analysis	TP _{cg-tot}	TP _{cg-add}	TP _{str}	TP _{topic-r}	TP _{topic-m}	TP _{total}	TP _{add}	TP _{art}	TP _{search}	p-value	τ_b
APFD	Avg	0.796	0.816	0.796	0.692	0.762	0.769	0.791	0.637	0.787	7.73E-10	0.667
	HSD	B	B	B	B	B	B	A	B	A		
APFDc	Avg	0.686	0.724	0.681	0.586	0.661	0.670	0.708	0.640	0.706	0.132	
	HSD	A	A	A	A	A	A	A	A	A		

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFDc and average APFD.

TABLE 11

Results for the ANOVA and Tukey HSD Tests on the Average APFD and APFDc Values at Test-Method Level Across Smaller Subject Programs

Metrics	Analysis	TP _{cg-tot}	TP _{cg-add}	TP _{str}	TP _{topic-r}	TP _{topic-m}	TP _{total}	TP _{add}	TP _{art}	TP _{search}	p-value	τ_b
APFD	Avg	0.751	0.799	0.799	0.771	0.804	0.797	0.885	0.791	0.878	2.29E-15	0.444
	HSD	B	B	B	B	B	B	A	B	A		
APFDc	Avg	0.604	0.700	0.670	0.655	0.673	0.605	0.657	0.645	0.696	0.572	
	HSD	A	A	A	A	A	A	A	A	A		

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFDc and average APFD.

TABLE 12

Results for the ANOVA and Tukey HSD Tests on the Average APFD and APFDc Values at Test-Method Level Across Larger Subject Programs

Metrics	Analysis	TP _{cg-tot}	TP _{cg-add}	TP _{str}	TP _{topic-r}	TP _{topic-m}	TP _{total}	TP _{add}	TP _{art}	TP _{search}	p-value	τ_b
APFD	Avg	0.777	0.837	0.827	0.791	0.829	0.821	0.912	0.805	0.892	3.21E-12	0.333
	HSD	C	BC	C	C	C	C	A	C	AB		
APFDc	Avg	0.673	0.774	0.671	0.702	0.686	0.660	0.759	0.692	0.774	0.086	
	HSD	A	A	A	A	A	A	A	A	A		

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFDc and average APFD.

levels, respectively. In each figure, each row presents both the APFD and APFDc results on the corresponding subject. In each sub-figure, the x -axis presents the versions used as the old version during software evolution (note that the most recent versions are always used as the new version during software evolution), while the y -axis presents the APFD or APFDc values. We show the APFD or APFDc distributions of different technique using box-plots of different colors, where

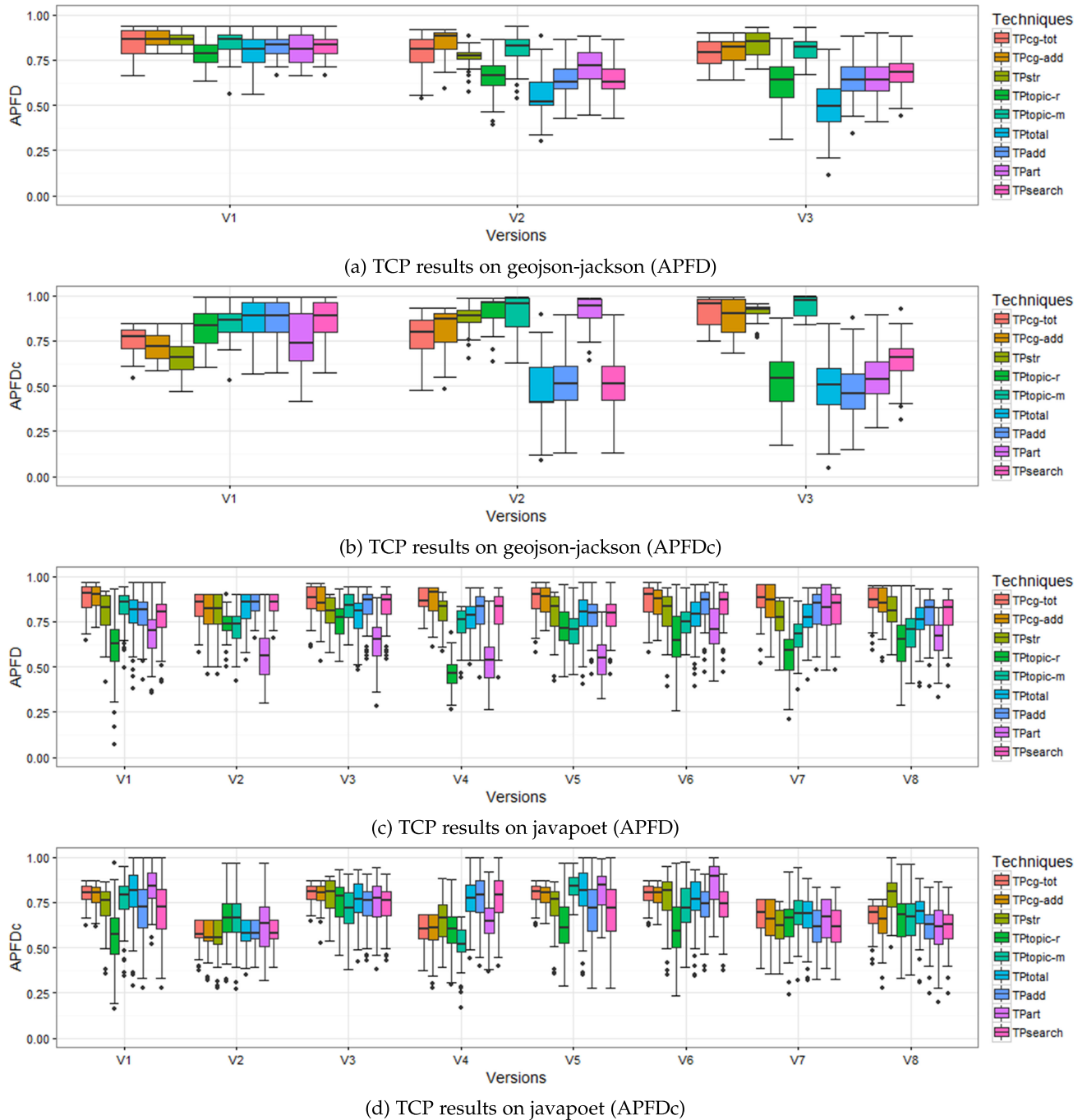


Fig. 4. Test-Class-level test prioritization in evolution.

the boxes represent the 25th to 75th percentiles, the centerlines represent the median values, and the dots represent the outlier points. Due to the limited space, we only show the results of two subject programs. The results of all twelve subject programs can be found in our online appendix [48], available in the online supplemental material. Following prior work [46], using different versions as the old version during software evolution allows us to understand the impact of software evolution on TCP in details. To illustrate, for a project with n versions, where ($n > 2$), we will have a set of $n - 1$ results for applying each studied TCP technique. That is, running TCPs on older program versions and then applying the prioritized set of test cases on the faulty variants of the most recent version (i.e., the latest versions with mutants) allows us to

understand the performance of TCP techniques in an evolutionary scenario. Note that more recent project versions may have tests not included in older project versions; we ignore such tests since the studied techniques would not be able to prioritize those tests based on old project versions.

If software evolution impacts TCP effectiveness, using earlier program versions for test prioritization would likely be less effective than using more up-to-date versions for test prioritization due to code changes. In other words, APFD/APFDc values should increase when using newer versions for prioritization. However, we observe no such trend for either APFD or APFDc for any TCP technique on any studied subject at the test-class or test-method level. This observation confirms prior work [28], [46] that code changes do

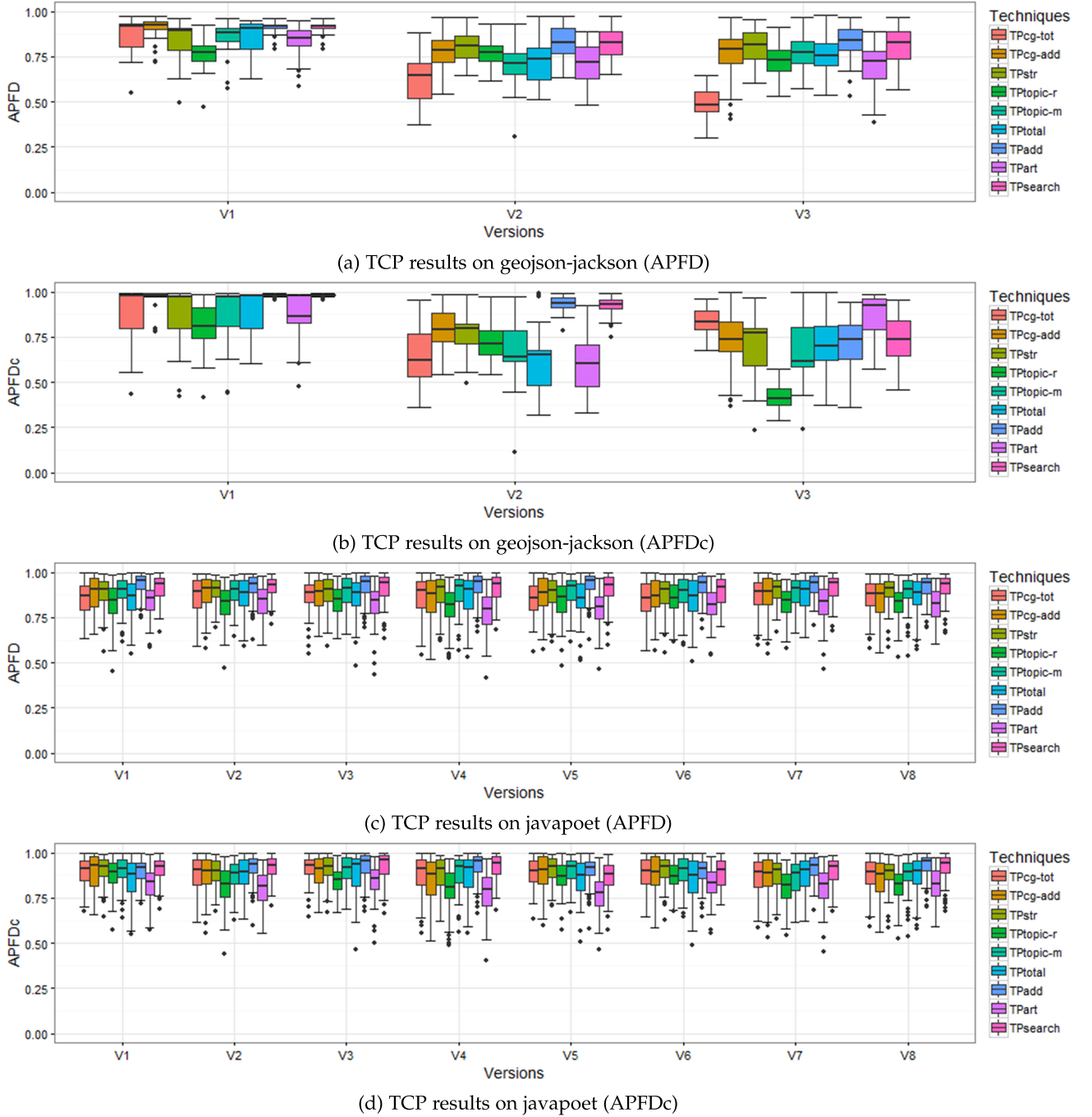


Fig. 5. Test-Method-level test prioritization in evolution.

not impact the effectiveness of dynamic TCP techniques in terms of APFD. Furthermore, our work is the first to illustrate that the same finding holds for static TCP techniques as well as the more practical APFDc metric. These results most likely arise due to the fact that all studied TCP techniques approximate fault detection capabilities based on a certain set of criteria (such as call graphs, textual information, or code coverage), and software evolution usually does not result in large relative changes between commits for these different criteria (e.g., some tests may always have higher code coverage throughout project evolution).

We also find that the performance comparison in terms of APFD between dynamic and static TCP techniques is not impacted by software evolution. For instance, static TCP

techniques tend to outperform dynamic TCP techniques in terms of both APFD at the test-class granularity on most subjects, while dynamic TCP techniques tend to outperform static TCP techniques in terms of APFD at the test-method granularity on most subjects. APFDc values tend to exhibit more variance during software evolution. For example, for the *javapoet* subject at test-class level, the static TP_{str} technique outperforms all other techniques when using V_8 information to prioritize tests for V_8 , while the dynamic TP_{art} technique performs the best when using V_1 information to prioritize tests for V_8 . One potential explanation for this observation is that tests with similar fault detection capabilities may have totally different execution times during evolution, causing high variances between APFDc values.

TABLE 13
Results for Average APFD Values on Different Sizes of Mutation Faults

Granularity	Sizes	TP _{cg-tot}	TP _{cg-add}	TP _{str}	TP _{topic-r}	TP _{topic-m}	TP _{total}	TP _{add}	TP _{art}	TP _{search}	τ_b
Test-class	1	0.772	0.786	0.775	0.67	0.741	0.737	0.771	0.639	0.767	0.944
	2	0.776	0.788	0.775	0.671	0.745	0.738	0.771	0.639	0.768	1
	3	0.776	0.788	0.777	0.673	0.746	0.738	0.769	0.638	0.766	0.944
	4	0.777	0.789	0.777	0.675	0.745	0.739	0.771	0.637	0.768	0.944
	5	0.777	0.789	0.778	0.675	0.746	0.739	0.771	0.635	0.767	0.944
	6	0.777	0.79	0.777	0.675	0.746	0.738	0.77	0.634	0.767	1
	7	0.778	0.79	0.777	0.675	0.746	0.738	0.769	0.633	0.766	1
	8	0.778	0.79	0.777	0.676	0.746	0.738	0.77	0.633	0.766	1
	9	0.778	0.79	0.777	0.676	0.746	0.738	0.769	0.633	0.766	1
	10	0.778	0.79	0.777	0.676	0.747	0.738	0.77	0.634	0.766	1
Test-method	1	0.759	0.82	0.813	0.777	0.814	0.807	0.901	0.798	0.885	1
	2	0.763	0.82	0.816	0.781	0.818	0.809	0.902	0.802	0.887	1
	3	0.763	0.818	0.814	0.78	0.815	0.81	0.9	0.8	0.886	1
	4	0.764	0.819	0.814	0.782	0.817	0.811	0.901	0.802	0.887	1
	5	0.764	0.818	0.814	0.782	0.818	0.811	0.9	0.8	0.887	1
	6	0.762	0.817	0.813	0.781	0.816	0.809	0.9	0.8	0.886	1
	7	0.763	0.817	0.813	0.781	0.816	0.81	0.9	0.8	0.886	1
	8	0.763	0.818	0.812	0.781	0.816	0.81	0.899	0.8	0.886	1
	9	0.763	0.818	0.812	0.781	0.816	0.81	0.899	0.799	0.885	1
	10	0.764	0.818	0.813	0.781	0.816	0.809	0.899	0.799	0.885	1

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFD values with different sizes of mutation faults and the average APFD values shown in Tables 5 and 6.

RQ₅: On average, software evolution does not have a clear impact on the measured effectiveness of the studied TCP techniques. Corroborating results of RQ₁ and RQ₂, we find that the APFD values for techniques tend to exhibit lower variance than APFDc values.

4.4 Impact of Mutant Quantities on TCP Effectiveness

Prior works examining TCP generally directly seed a certain number of faults to form a faulty version (or groups of faulty versions) to investigate TCP effectiveness according

to the APFD or APFDc, similar to the setup used in our study to answer RQ₁-RQ₃. However, we wish to further analyze the impacts of the quantity of mutants utilized in experimental settings and whether or not this impacts the effectiveness of techniques. The experimental results for APFD and APFDc are shown Tables 13 and 14, respectively. In each table, Column 1 lists the test-case granularities studied, column 2 lists the number of mutants seeded into each faulty version/group, columns 3-11 present the APFD/APFDc results for each studied technique, and finally the last column presents the Kendall τ_b Rank Correlation Coefficient between the average values with each fault quantity

TABLE 14
Results for Average APFDc Values on Different Sizes of Mutation Faults

Granularity	Sizes	TP _{cg-tot}	TP _{cg-add}	TP _{str}	TP _{topic-r}	TP _{topic-m}	TP _{total}	TP _{add}	TP _{art}	TP _{search}	τ_b
Test-class	1	0.655	0.681	0.668	0.572	0.658	0.619	0.653	0.622	0.652	0.944
	2	0.653	0.679	0.668	0.571	0.66	0.616	0.652	0.621	0.652	0.944
	3	0.653	0.679	0.669	0.574	0.66	0.617	0.651	0.619	0.65	0.944
	4	0.652	0.678	0.667	0.575	0.657	0.617	0.652	0.615	0.651	1
	5	0.652	0.678	0.668	0.575	0.659	0.616	0.651	0.614	0.651	1
	6	0.653	0.68	0.667	0.575	0.658	0.616	0.65	0.613	0.65	1
	7	0.653	0.68	0.667	0.575	0.658	0.616	0.65	0.613	0.65	1
	8	0.654	0.681	0.667	0.576	0.659	0.617	0.651	0.613	0.65	1
	9	0.654	0.68	0.668	0.576	0.659	0.617	0.651	0.613	0.65	1
	10	0.654	0.68	0.668	0.576	0.659	0.616	0.651	0.613	0.65	1
Test-method	1	0.637	0.745	0.671	0.681	0.681	0.634	0.715	0.675	0.739	0.889
	2	0.639	0.739	0.675	0.683	0.684	0.634	0.714	0.675	0.739	0.944
	3	0.639	0.738	0.672	0.681	0.679	0.634	0.711	0.672	0.738	0.889
	4	0.638	0.737	0.671	0.68	0.68	0.634	0.711	0.672	0.737	0.944
	5	0.638	0.737	0.672	0.68	0.681	0.635	0.711	0.671	0.738	0.944
	6	0.636	0.738	0.671	0.678	0.68	0.634	0.711	0.671	0.737	0.944
	7	0.637	0.738	0.671	0.679	0.68	0.635	0.711	0.671	0.737	1
	8	0.638	0.739	0.67	0.679	0.68	0.635	0.71	0.671	0.737	0.944
	9	0.638	0.738	0.67	0.679	0.679	0.635	0.71	0.67	0.737	0.944
	10	0.638	0.737	0.67	0.678	0.679	0.634	0.709	0.67	0.736	1

This table follows the same format as Table 13.

TABLE 15
Results for Average APFD Values on Different Types of Mutation Faults

Gra.	Types	TP _{cg-tot}	TP _{cg-add}	TP _{str}	TP _{topic-r}	TP _{topic-m}	TP _{total}	TP _{add}	TP _{art}	TP _{search}	τ_b
Class	NegateConditionals	0.785	0.796	0.792	0.699	0.762	0.749	0.788	0.677	0.784	0.889
	RemoveConditional	0.792	0.803	0.794	0.703	0.764	0.755	0.792	0.679	0.788	0.944
	ConstructorCall	0.784	0.797	0.787	0.682	0.757	0.74	0.772	0.663	0.766	0.944
	NonVoidMethodCall	0.774	0.783	0.774	0.668	0.74	0.731	0.761	0.626	0.756	1
	Math	0.782	0.787	0.776	0.685	0.748	0.706	0.775	0.657	0.772	1
	MemberVariable	0.798	0.816	0.772	0.69	0.752	0.776	0.794	0.676	0.79	0.778
	InlineConstant	0.76	0.778	0.776	0.687	0.752	0.707	0.752	0.641	0.75	0.889
	Increments	0.791	0.815	0.792	0.725	0.784	0.726	0.8	0.7	0.796	0.722
	ArgumentPropagation	0.775	0.787	0.775	0.676	0.751	0.738	0.769	0.624	0.766	1
	ConditionalsBoundary	0.787	0.81	0.809	0.709	0.776	0.737	0.78	0.69	0.778	0.944
	Switch	0.859	0.838	0.882	0.822	0.856	0.849	0.867	0.757	0.864	0.5
	VoidMethodCall	0.782	0.781	0.771	0.659	0.72	0.757	0.749	0.623	0.748	0.778
	InvertNegs	0.744	0.805	0.849	0.669	0.738	0.63	0.757	0.726	0.743	0.667
	ReturnVals	0.781	0.802	0.779	0.671	0.748	0.732	0.762	0.651	0.759	1
	RemoveIncrements	0.755	0.797	0.761	0.684	0.738	0.685	0.762	0.645	0.759	0.778
Method	NegateConditionals	0.787	0.842	0.845	0.809	0.851	0.829	0.921	0.828	0.911	0.889
	RemoveConditional	0.792	0.846	0.848	0.813	0.852	0.834	0.925	0.832	0.914	0.889
	ConstructorCall	0.756	0.816	0.8	0.782	0.806	0.791	0.886	0.808	0.873	0.833
	NonVoidMethodCall	0.755	0.809	0.811	0.766	0.815	0.804	0.891	0.783	0.874	0.889
	Math	0.745	0.801	0.789	0.776	0.795	0.777	0.902	0.809	0.882	0.778
	MemberVariable	0.799	0.858	0.837	0.809	0.843	0.838	0.914	0.832	0.905	0.944
	InlineConstant	0.735	0.777	0.785	0.766	0.788	0.774	0.881	0.794	0.869	0.667
	Increments	0.771	0.853	0.857	0.835	0.865	0.81	0.942	0.854	0.926	0.722
	ArgumentPropagation	0.752	0.821	0.824	0.768	0.829	0.81	0.899	0.793	0.881	0.889
	ConditionalsBoundary	0.761	0.825	0.827	0.806	0.833	0.809	0.901	0.819	0.89	0.833
	Switch	0.86	0.881	0.932	0.871	0.946	0.902	0.968	0.892	0.952	0.778
	VoidMethodCall	0.771	0.805	0.792	0.766	0.8	0.81	0.871	0.779	0.862	0.778
	InvertNegs	0.752	0.843	0.813	0.69	0.784	0.726	0.849	0.799	0.812	0.611
	ReturnVals	0.766	0.842	0.82	0.785	0.819	0.792	0.889	0.803	0.876	0.889
	RemoveIncrements	0.737	0.84	0.861	0.807	0.868	0.777	0.935	0.846	0.907	0.722

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFD values with different types of mutation faults and the average APFD values shown in Tables 5 and 6.

and our default settings (shown in Tables 5 and 6). From the tables, we make the following observations. For both APFD and APFDc values, the mutant quantity does not dramatically impact the results for all of the studied techniques. For example, at the test-class level, the average APFD values of TP_{cg-add} range from 0.786 to 0.790 for all the studied fault quantity settings, while its APFDc values range from 0.678 to 0.681. This finding indicates that the effectiveness of the studied techniques when seeding any number of mutants into each group will be roughly equivalent, demonstrating the validity of the mutant seeding processes of prior TCP work [27], [46], [50], [73]. The largest impact that fault quantities had were for the APFDc metric at the test-method level. The likely reason for this is that the test-method level techniques prioritize tests at a finer granularity, and thus are more sensitive to the impact of execution time. For example, APFDc of fault groups with only one fault in each group only considers the time to detect only the first fault (while APFDc of fault groups with 5 faults in each group considers the time to detect all the 5 faults), leading to the higher variance.

RQ₆: The quantity of mutants used, as stipulated in the experimental settings of mutation analysis-based evaluations of TCP approaches, does not significantly impact the effectiveness of TCP techniques in terms of either APFD or APFDc, demonstrating the validity of the fault seeding process of prior work in this context.

4.5 Impact of Mutant Types on TCP Effectiveness

In this RQ, we further investigate whether different mutant types may impact TCP results in terms of either APFD and APFDc. The experimental results for APFD and APFDc are shown Tables 15 and 16, respectively. In each table, column 1 lists the test-case granularities studied, column 2 lists the different types of mutants seeded into each faulty version, columns 3-11 present the APFD/APFDc results for each studied technique, and finally the last column presents the Kendall τ_b Rank Correlation Coefficient between the average APFD/APFDc values with each fault type and our default settings (shown in Tables 5 and 6). From the tables, we can make the following observations. Overall, the vast majority of the studied mutant types tend to have a medium to high coefficient (i.e., the range of Kendall correlation coefficient values is from 0.5 to 1.0). This implies that the performance of TCP techniques when applied to detecting only certain mutant types highly correlates to the performance observed when applied to detecting all mutants. This finding indicates that the findings in prior work on TCP (including this work) generally hold across mutants seeded with differing mutation operators. Second, we also observe that there are several mutant types with low correlation with our default fault seeding, e.g., the Invert Negs Mutator and the Switch Mutator have the lowest correlation in both studied test granularities for both APFD and APFDc. Upon further investigation, we found one likely explanation to be the small number of mutants generated by such mutators. For example, the number of Invert Negs Mutator is quite small as

TABLE 16
Results for Average APFDc Values on Different Types of Mutation Faults

Gra.	Types	TP _{cg-tot}	TP _{cg-add}	TP _{str}	TP _{topic-r}	TP _{topic-m}	TP _{total}	TP _{add}	TP _{art}	TP _{search}	τ_b
Class	NegateConditionals	0.646	0.67	0.671	0.591	0.666	0.614	0.653	0.639	0.652	0.778
	RemoveConditional	0.655	0.677	0.673	0.594	0.668	0.62	0.659	0.64	0.658	0.833
	ConstructorCall	0.651	0.682	0.672	0.575	0.664	0.609	0.644	0.627	0.64	0.944
	NonVoidMethodCall	0.642	0.665	0.652	0.556	0.645	0.598	0.628	0.593	0.625	1
	Math	0.658	0.66	0.653	0.576	0.636	0.559	0.635	0.611	0.634	0.778
	MemberVariable	0.668	0.699	0.656	0.587	0.66	0.649	0.671	0.641	0.667	0.556
	InlineConstant	0.62	0.647	0.651	0.57	0.644	0.567	0.617	0.607	0.619	0.778
	Increments	0.636	0.663	0.655	0.584	0.657	0.584	0.639	0.633	0.639	0.667
	ArgumentPropagation	0.655	0.668	0.653	0.571	0.66	0.613	0.644	0.593	0.644	0.889
	ConditionalsBoundary	0.64	0.673	0.674	0.591	0.655	0.604	0.643	0.639	0.647	0.722
	Switch	0.806	0.775	0.803	0.663	0.779	0.736	0.763	0.822	0.759	0.333
	VoidMethodCall	0.628	0.631	0.624	0.519	0.602	0.606	0.602	0.579	0.605	0.611
	InvertNegs	0.537	0.579	0.705	0.498	0.666	0.472	0.746	0.747	0.714	0
	ReturnVals	0.652	0.685	0.661	0.56	0.649	0.603	0.636	0.619	0.635	0.889
	RemoveIncrements	0.605	0.639	0.664	0.546	0.628	0.544	0.602	0.649	0.602	0.5
Method	NegateConditionals	0.647	0.749	0.691	0.699	0.7	0.637	0.724	0.694	0.757	0.889
	RemoveConditional	0.653	0.753	0.696	0.704	0.706	0.643	0.73	0.701	0.762	0.889
	ConstructorCall	0.632	0.728	0.651	0.667	0.658	0.621	0.698	0.68	0.715	0.778
	NonVoidMethodCall	0.617	0.716	0.653	0.648	0.664	0.619	0.688	0.637	0.709	0.889
	Math	0.631	0.723	0.655	0.679	0.683	0.581	0.722	0.709	0.748	0.778
	MemberVariable	0.673	0.773	0.701	0.708	0.703	0.657	0.731	0.715	0.761	0.778
	InlineConstant	0.598	0.683	0.622	0.654	0.636	0.573	0.67	0.659	0.699	0.722
	Increments	0.606	0.739	0.671	0.706	0.695	0.611	0.731	0.713	0.763	0.667
	ArgumentPropagation	0.609	0.709	0.662	0.646	0.674	0.611	0.698	0.64	0.713	0.833
	ConditionalsBoundary	0.605	0.716	0.641	0.682	0.661	0.612	0.695	0.68	0.73	0.722
	Switch	0.752	0.787	0.786	0.78	0.796	0.768	0.812	0.777	0.839	0.722
	VoidMethodCall	0.626	0.683	0.615	0.619	0.63	0.599	0.635	0.613	0.666	0.833
	InvertNegs	0.7	0.816	0.6	0.537	0.655	0.651	0.66	0.713	0.752	0.389
	ReturnVals	0.63	0.742	0.661	0.671	0.669	0.608	0.687	0.664	0.711	0.889
	RemoveIncrements	0.597	0.752	0.699	0.697	0.726	0.575	0.709	0.73	0.743	0.667

The last column shows the results for Kendall tau Rank Correlation Coefficient τ_b between the average APFDc values with different types of mutation faults and the average APFDc values shown in Tables 5 and 6.

compared to other type of mutation faults (since it is only applicable to the cases of negative numbers), thus the results are dramatically different as compared to the results of mutation faults with the default setting. The Switch Mutator also has small Kendall correlation coefficient values as compared to other mutators. This is due to the fact that like the Invert Negs Mutator – the number of Switch Mutator faults is quite small as compared to other type of mutation faults (since it is only applicable to the cases of switches, which are not intensively used in common programs). Thus, the results are dramatically different as compared to the results of mutation faults with the default setting. Furthermore, we also observe that some mutation faults are more subtle than others. For example, the mutation faults created by Invert Negs Mutator tend to be more subtle than other types of mutation faults. For example, the Invert Negs Mutator operator simply inverts negation of an integer and floating point number (e.g., changing “return -i;” into “return i;”), while Non-Void Method Call Mutator or Void Method Call Mutator directly removes an entire method invocation. The subtle mutation faults introduced by Invert Negs Mutator can be harder to detect, making various static and dynamic techniques perform worse on those faults since the coverage or call graph information won’t provide precise guide for detecting such subtle faults.

RQ₇: The mutation operators used to seed faults, as stipulated in the experimental settings of mutation analysis-based evaluations of TCP approaches, do not significantly impact the effectiveness of TCP techniques in terms of either APFD or APFDc, demonstrating the comparative validity of the fault seeding process of prior work in this context.

4.6 Similarity Between Uncovered Faults for Different TCP Techniques

The overall results for the fault similarity analysis are shown in Fig. 6. The two figures represent the results comparing the average Jaccard similarity of the studied static techniques to the studied dynamic techniques for all subject programs across 500 randomly sampled faults at different prioritization cut points. These results indicate that there is only a small amount of similarity between these two classifications of techniques at the higher level cut points. More specifically, for test-method level, only ≈ 30 percent of the detected faults are similar between the two types of techniques for the top 10 percent of the prioritized test cases, and at test-class level only about ≈ 25 percent are similar for the top 10 percent of prioritized test cases. This result illustrates one of the key findings of this study: The studied static and dynamic TCP techniques do not uncover similar program faults at the top cut points of prioritized test cases. The

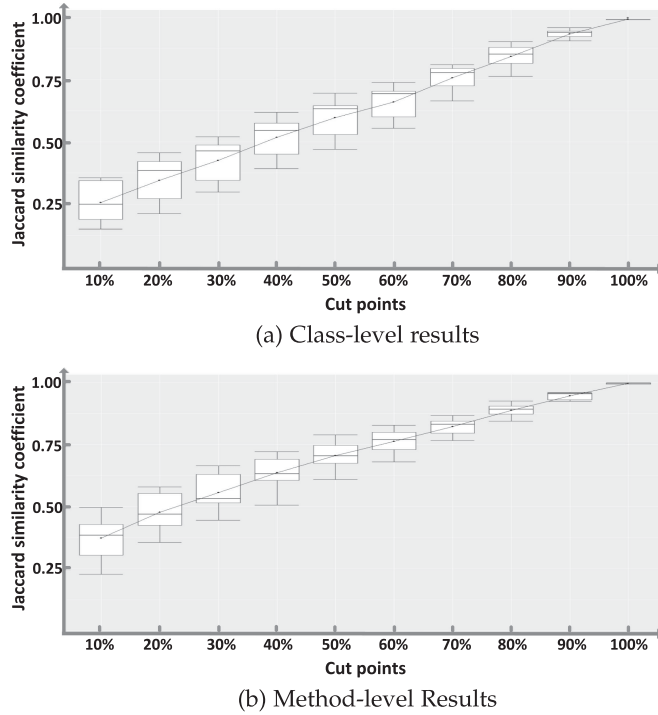


Fig. 6. Average Jaccard similarity of faults detected between static and dynamic techniques across all subjects at method and class-level granularity.

potential reason for these results is that different techniques use different types of information to prioritize test cases. For example, the studied static techniques typically aim to promote diversity between prioritized test cases using similarity/diversity metrics such as textual distance or

call-graph information. In contrast, the studied dynamic TCPs consider statement-level dynamic coverage to prioritize test cases. This finding raises interesting questions for future work regarding the possibility of combining static and dynamic information and the relative *importance* of faults that differing techniques might uncover. It should be noted that different coverage granularities for dynamic TCPs may also effect the results of similarity, however we leave such an investigation for future work. From these figures we can also conclude that the techniques are slightly more similar at method level than at class level.

To further illustrate this point we calculated the Jaccard coefficients for each pair of TCPs for each subject program, and show the results in Tables 17 and 18. For each pair of techniques we group the subjects into the categories described in Section 3. Due to space limitations, we only show results for the top 10 and top 50 percent of prioritized test-cases, a complete dataset can be found at [48]. The results confirm the conclusions drawn from Fig. 6. It is clear that when comparing the studied static and dynamic techniques, more subjects are classified into the *highly-dissimilar* and *dissimilar* categories at the top 10 percent on both of test-method and test-class levels. Another relevant conclusion that can be made is that the dissimilarity between techniques is not universal across all subjects. That is, even though two techniques may be dissimilar across several subjects, there are some cases where similarity still exists. This suggests that only certain types of programs that exhibit different characteristics may present the opportunity of performance improvement for TCPs by using both static and dynamic information. In addition, at the cut point for the top 50 percent of prioritized test cases, it is obvious that

TABLE 17
The Tables Show the Classification of Subjects on Different Granularities Using Jaccard Distance

(a) This table shows the classification of subjects at the cut point 10% on test-class level.																																				
	T1				T2				T3				T4				T5				T6				T7				T8				T9			
TP1	-	-	-	-	3	2	16	37	11	13	18	16	28	20	3	7	14	18	17	9	11	19	13	15	15	14	16	13	32	18	5	3	15	15	13	15
TP2	3	2	16	37	-	-	-	-	11	14	15	18	27	23	3	5	13	24	14	7	11	20	14	13	14	16	13	15	30	23	4	1	13	17	10	18
TP3	11	13	18	16	11	14	15	18	-	-	-	-	30	13	10	5	12	12	14	20	20	15	12	11	18	18	13	9	33	15	6	4	18	18	12	10
TP4	28	20	3	7	27	23	3	5	30	13	10	5	-	-	-	-	14	15	12	17	26	17	11	4	24	15	15	4	31	13	7	7	24	16	13	5
TP5	14	18	17	9	13	24	14	7	12	12	14	20	14	15	12	17	-	-	-	-	19	24	7	8	21	20	10	7	30	16	7	5	21	20	9	8
TP6	11	19	13	15	11	20	14	13	20	15	12	11	26	17	11	4	19	24	7	8	-	-	-	-	2	13	11	32	28	13	9	8	2	12	14	30
TP7	15	14	16	13	14	16	13	15	18	18	13	9	24	15	15	4	21	20	10	7	2	13	11	32	-	-	-	-	25	16	13	4	0	0	2	56
TP8	32	18	5	3	30	23	4	1	33	15	6	4	31	13	7	7	30	16	7	5	28	13	9	8	25	16	13	4	-	-	-	25	15	15	3	
TP9	15	15	13	15	13	17	10	18	18	18	12	10	24	16	13	5	21	20	9	8	2	12	14	30	0	0	2	56	25	15	15	3	-	-	-	-
Total	129	119	101	115	122	139	89	114	153	118	100	93	204	132	74	54	144	149	90	81	119	133	91	121	119	112	93	140	234	129	66	35	118	113	88	145

(b) This table shows the classification of subjects at the cut point 10% on test-method level.																																				
	T1				T2				T3				T4				T5				T6				T7				T8				T9			
TP1	-	-	-	-	3	14	28	13	11	23	17	7	11	29	15	3	12	20	20	6	6	14	19	19	6	19	22	11	21	23	11	3	3	21	20	14
TP2	3	14	28	13	-	-	-	-	7	18	24	9	5	22	26	5	6	19	28	5	1	21	25	11	2	14	23	19	14	21	19	4	3	14	25	16
TP3	11	23	17	7	7	18	24	9	-	-	-	-	4	21	27	6	0	3	17	38	7	16	23	12	4	12	26	16	15	22	20	1	5	12	29	12
TP4	11	29	15	3	5	22	26	5	4	21	27	6	-	-	-	-	6	22	26	4	7	27	21	3	5	26	23	4	12	25	19	2	4	26	22	6
TP5	12	20	20	6	6	19	28	5	0	3	17	38	6	22	26	4	-	-	-	-	7	17	24	10	6	5	34	13	13	24	20	1	7	7	34	10
TP6	6	14	19	19	1	21	25	11	7	16	23	12	7	27	21	3	7	17	24	10	-	-	-	1	11	29	17	19	21	16	2	2	11	26	19	
TP7	6	19	22	11	2	14	23	19	4	12	26	16	5	26	23	4	6	5	34	13	1	11	29	17	-	-	-	-	10	19	26	3	1	3	6	48
TP8	21	23	11	3	14	21	19	4	15	22	20	1	12	25	19	2	13	24	20	1	19	21	16	2	10	19	26	3	-	-	-	13	17	23	5	
TP9	3	21	20	14	3	14	25	16	5	12	29	12	4	26	22	6	7	7	34	10	2	11	26	19	1	3	6	48	13	17	23	5	-	-	-	-
Total	73	163	152	76	41	143	198	82	53	127	183	101	54	198	179	33	57	117	203	87	50	138	183	93	35	109	189	131	117	172	154	21	38	111	185	130

The four values in each cell are the numbers of subject projects, the faults of which detected by two techniques are highly dissimilar, dissimilar, similar and highly similar respectively. The technique enumeration is consistent with Table 7.

Authorized licensed use limited to: AIT Austrian Institute of Technology. Downloaded on August 08, 2026 at 18:53:33 UTC from IEEE Xplore. Restrictions apply.

TABLE 18
The Tables Show the Classification of Subjects on Different Granularities Using Jaccard Distance

(a) This table shows the classification of subjects at the cut point 50% on test-class level.																																					
	T1				T2				T3				T4				T5				T6				T7				T8				T9				
TP1	-	-	-	-	0	2	10	46	0	1	10	47	1	6	30	21	1	3	17	37	0	6	17	35	0	2	22	34	2	9	29	18	0	2	22	34	
TP2	0	2	10	46	-	-	-	-	1	0	9	48	0	6	28	24	0	2	17	39	0	2	23	33	0	1	17	40	2	7	24	25	0	1	16	41	
TP3	0	1	10	47	1	0	9	48	-	-	-	-	1	6	23	28	0	2	11	45	1	4	19	34	1	1	16	40	2	9	21	26	1	1	16	40	
TP4	1	6	30	21	0	6	28	24	1	6	23	28	-	-	-	-	0	7	20	31	0	7	32	19	1	3	29	25	4	9	19	26	1	3	29	25	
TP5	1	3	17	37	0	2	17	39	0	2	11	45	0	7	20	31	-	-	-	-	0	7	20	31	0	4	17	37	3	7	25	23	0	4	18	36	
TP6	0	6	17	35	0	2	23	33	1	4	19	34	0	7	32	19	0	7	20	31	-	-	-	-	0	2	18	38	2	15	28	13	0	2	18	38	
TP7	0	2	22	34	0	1	17	40	1	1	16	40	1	3	29	25	0	4	17	37	0	2	18	38	-	-	-	-	2	11	13	32	0	0	0	58	
TP8	2	9	29	18	2	7	24	25	2	9	21	26	4	9	19	26	3	7	25	23	2	15	28	13	2	11	13	32	-	-	-	-	2	10	13	33	
TP9	0	2	22	34	0	1	16	41	1	1	16	40	1	3	29	25	0	4	18	36	0	2	18	38	0	0	0	58	2	10	13	33	-	-	-	-	
Total	4	31	157	272	3	21	144	296	7	24	125	308	8	47	210	199	4	36	145	279	3	45	175	241	4	24	132	304	19	77	172	196	4	23	132	305	

(b) This table shows the classification of subjects at the cut point 50% on test-method level.																																					
	T1				T2				T3				T4				T5				T6				T7				T8				T9				
TP1	-	-	-	-	1	0	20	37	0	3	20	35	1	1	31	25	1	3	17	37	0	1	13	44	0	1	17	40	1	1	23	33	0	1	17	40	
TP2	1	0	20	37	-	-	-	-	0	1	10	47	0	1	16	41	0	0	15	43	0	2	16	40	0	0	9	49	0	0	12	46	0	0	10	48	
TP3	0	3	20	35	0	1	10	47	-	-	-	-	0	2	18	38	0	0	3	55	0	1	12	45	0	0	6	52	0	1	10	47	0	0	8	50	
TP4	1	1	31	25	0	1	16	41	0	2	18	38	-	-	-	-	0	2	12	44	0	2	20	36	0	0	13	45	0	0	14	44	0	0	14	44	
TP5	1	3	17	37	0	0	15	43	0	0	3	55	0	2	12	44	-	-	-	-	0	0	16	42	0	0	5	53	0	1	7	50	0	0	6	52	
TP6	0	1	13	44	0	2	16	40	0	1	12	45	0	2	20	36	0	0	16	42	-	-	-	-	0	0	10	48	0	0	19	39	0	0	9	49	
TP7	0	1	17	40	0	0	9	49	0	0	6	52	0	0	13	45	0	0	5	53	0	0	10	48	-	-	-	-	0	0	3	55	0	0	2	56	
TP8	1	1	23	33	0	0	12	46	0	1	10	47	0	0	14	44	0	1	7	50	0	0	19	39	0	0	3	55	-	-	-	-	0	0	2	56	
TP9	0	1	17	40	0	0	10	48	0	0	8	50	0	0	14	44	0	0	6	52	0	0	9	49	0	0	2	56	0	0	2	56	-	-	-	-	
Total	4	11	158	291	1	4	108	351	0	8	87	369	1	8	138	317	1	6	81	376	0	6	115	343	0	1	65	398	1	3	90	370	0	1	68	395	

The four values in each cell are the numbers of subject projects, the faults of which detected by two techniques are highly dissimilar, dissimilar, similar and highly similar respectively. The technique enumeration is consistent with Table 7.

fewer subjects are classified into the *highly-dissimilar* and *dissimilar* categories. This is not surprising, because as the cut point increases the different techniques tend to discover more faults, limiting the potential for variance.

There are two potential reasons why we might observe higher numbers of *dissimilar* faults detected at the highest cut points: 1) different types of mutants are being detected; and 2) mutants of the same type in different locations are being detected. To investigate whether our observations are due to different fault types, we examine the counts and the percentages for different types of mutants that are detected by top 10 percent test cases at both the test-class and test-method level. The results are shown in Figs. 7 and 8. When observing that the ratio for different types of mutation faults detected by different TCP techniques, we find that, as a whole, all TCP techniques detect a similar ratio of each mutant type, implying that mutant type is generally not the cause for the dissimilar faults at the higher cut points, but rather, mutants of the same type

present different locations in source code are the more likely explanation.

RQs: The studied static and dynamic TCP techniques tend to discover dissimilar faults for the most highly prioritized test cases. Specifically, at the test-method level static and dynamic techniques agree only on ≈ 35 percent of uncovered faults for the top 10 percent of prioritized test cases. Additionally, a subset of subjects exhibit higher levels of detected fault similarity, suggesting that only software systems with certain characteristics may benefit from differing TCP approaches. Furthermore, the most highly prioritized test cases by different TCP techniques share similar capabilities in detecting different types of mutation faults.

4.7 Efficiency of Static TCP Techniques

The results of time costs for the studied static techniques at both of test-method and test-class levels are shown in

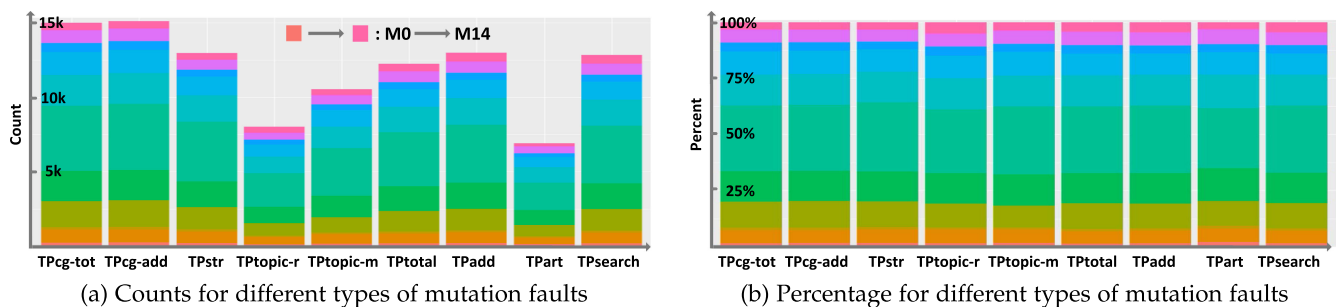


Fig. 7. Counts and percentage for different types of mutation faults across all subjects at cut point 10 percent for class-level granularity. The types of mutation faults are classified based on the mutation operators shown in Table 3.

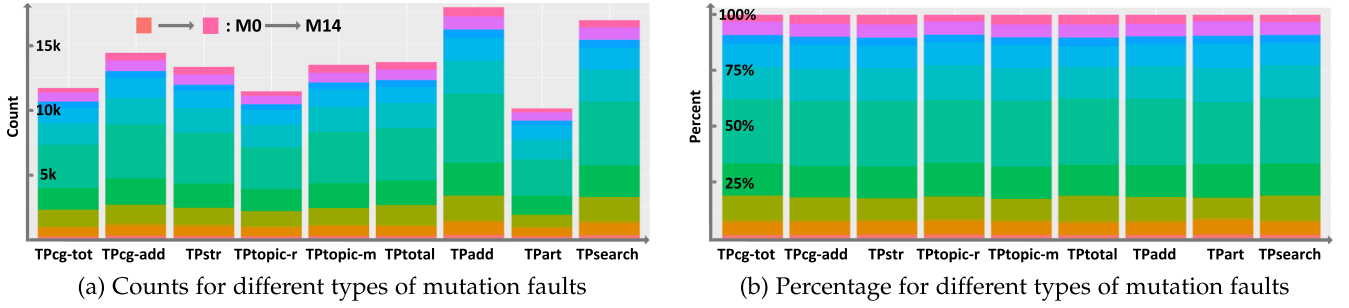


Fig. 8. Counts and percentage for different types of mutation faults across all subjects at cut point 10 percent for method-level granularity. The types of mutation faults are classified based on the mutation operators shown in Table 3.

Table 19. Note that, the time of pre-processing for TP_{cg-tot} and TP_{cg-add} are the same for both method and class levels. As the table shows, all studied techniques require similar time to pre-process the data at both method and class levels and to rank test cases on class level. But the times for prioritization are quite different at method level. We find that TP_{cg-tot} and TP_{cg-add} take much less time to prioritize test cases (totaling 23.78 seconds and 37.02 seconds), as compared to TP_{str} (totaling 78,835.97 seconds), $TP_{topic-r}$ (totaling 48,310.93 seconds) and $TP_{topic-m}$ (totaling 15,573.71 seconds). In particular, the following three techniques, TP_{str} , $TP_{topic-r}$, and $TP_{topic-m}$ take much longer time on some subjects (e.g., $P53$ and $P58$). These subjects have a large number of test cases (see Table 2), implying that TP_{str} , $TP_{topic-r}$ and $TP_{topic-m}$ will take more time as the number of test cases increases. Overall, all techniques take a reasonable amount of time to preprocess data and prioritize test cases. At test-method level, TP_{cg-tot} and TP_{cg-add} are much more efficient. TP_{str} , $TP_{topic-r}$ and $TP_{topic-m}$ require more time to prioritize increasing numbers of test cases.

RQ₉: On test-method level, TP_{cg-tot} and TP_{cg-add} are the more efficient in prioritizing test cases. TP_{str} , $TP_{topic-r}$ and $TP_{topic-m}$ take more time when the number of test cases increases. The time of pre-processing and prioritization on test class level for all static techniques are quite similar.

5 THREATS TO VALIDITY

Threats to Internal Validity. In our implementation, we used PIT to generate mutation faults to simulate real program faults. One potential threat is that the mutation faults may not reflect all “natural” characteristics of real faults. However, mutation faults have been widely used in the domain

of software engineering research and have, under proper circumstances, been demonstrated to be representative of the actual program faults [33]. Further threats related to mutation testing include the potential bias introduced by equivalent and trivial mutants. In the context of our experimental settings, equivalent mutants will not be detected by test cases. As explained in Section 3, we ignore all mutants that cannot be detected by test cases. Thus, we believe that this threat is sufficiently mitigated. To answer RQ_1 - RQ_3 , we randomly selected 500 faults (100 groups and five faults per group) for each subject system, which may impact the evaluation of TCP performance. However, this follows the guidelines and methodology of previous studies [46], [73], minimizing this threat. Additionally, we also introduce two research questions, RQ_4 and RQ_5 , to investigate the impact of mutant quantities and type on TCP evaluation, allowing us to examine the validity of past evaluations of TCP effectiveness. There is also a potential threat that due to trivial/subsumed mutants (e.g., those that are easily distinguished from the original program) outlined in recent work [28], [53]. The trivial or subsumed mutants may potentially impact the results (e.g., inflate the APFD values). However, we do not specifically control the trivial and subsumed mutants, following the body of previous work in the TCP area [46], [50], [73], since in practice real faults may also be trivial or subsume each other. In addition, we involve a large set of randomly selected mutants, further mitigating this threat to validity. We encourage future studies to further examine this potential threat.

To perform this study we reimplemented eight TCP techniques presented in prior work. It is possible that there may be some slight differences between the original authors’ implementations and our own. However, we performed this task closely following the technical details of the prior techniques and set parameters following the guidelines in

TABLE 19
Execution Costs for the Static TCP Techniques

Techniques	Pre-processing				Test Prioritization			
	Avg.	Min	Max	Sum	Avg.	Min	Max	Sum
TP_{cg-tot}	244.14/244.14	1.21/1.21	13785.86/13785.86	14159.97/14159.97	0.20/0.41	0/0	3.10/10.58	11.37/23.78
TP_{cg-add}	244.14/244.14	1.21/1.21	113785.86/13785.86	14159.97/14159.97	0.18/0.64	0/0	2.87/19.98	10.59/37.02
TP_{str}	0.35/0.37	0.04/0.04	2.95/2.41	20.04/21.63	4.03/1,359.24	0.01/0.02	115.82/57,134.30	233.76/78,835.97
$TP_{topic-r}$	0.41/1.55	0.03/0.09	3.81/14.80	24.99/89.63	0.15/832.95	0/0.01	1.72/40,594.66	8.50/48,310.93
$TP_{topic-m}$	1.51/3.79	0.13/0.22	12.10/50.14	87.76/219.93	0.19/268.51	0/0.07	1.98/10,925.26	10.95/15,573.71

The table lists the average, min, max, and sum of costs across all subject programs for both test-class level and test-method level (i.e., cost at test-class level/cost at test-method level). Time is measured in second.

Authorized licensed use limited to: AIT Austrian Institute of Technology. Downloaded on August 08, 2026 at 18:53:33 UTC from IEEE Xplore. Restrictions apply.

the original publications. Additionally, the authors of this paper met for and open code review regarding the studied approaches. Furthermore, based on our general findings, we believe our implementations to be accurate.

Threats to External Validity. The main external threat to our study is that we experimented on 58 software systems, which may impact the generalizability of the results. Involving more subject programs would make it easier to reason about how the studied TCP techniques would perform on software systems of different languages and purposes. However, we chose 58 systems with varying sizes (1.2KLoC - 83.0 KLoC) and different numbers of detectable mutants (132 - 46,429), which makes for a highly representative set of Java programs, more so than any past study. Additionally, some subjects were used as benchmarks in recent papers [58]. Thus, we believe our study parameters have sufficiently mitigated this threat to a point where useful and actionable conclusions can be drawn in the context of our research questions. In addition, we seeded mutants using operators provided in PIT. It is possible that having different types of operators or using different mutation analysis tool may impact the results of our study. However, PIT is one of the most popular mutation analysis tools and has been widely used in software testing research. Thus, we believe our design of the study has mitigated this threat. Finally, while it would be interesting to investigate the effectiveness of TCPs on detecting real regression faults, this is a difficult task. A large set of real regression faults is notoriously hard to collect in practice. The reason is that during real-world software development, the developers usually run regression tests before committing new revisions to the code repositories, and will fix any regression faults before the commits, leaving few real regression faults recorded in the code repositories. On the other hand, mutants have been shown to be suitable for simulating real faults for software-testing experimentation [3], [33] (including test-prioritization experimentation [15]). Furthermore, mutation testing is widely used in recent TCP research work [28], [46]. Thus, in this paper we evaluate TCP effectiveness in terms of detecting mutants, and leave the investigation of TCP performance on real regression faults as future work.

Finally, we selected four static TCP techniques to experiment with in our empirical study. There are some other recent works proposing static TCP techniques [6], [58], but we focus only on those which do not require additional inputs, such as code changes or requirements in this empirical study. Also, we only compared the static techniques with four state-of-art dynamic TCP techniques with statement-level coverage. We do not study the potential impact of different coverage granularities on dynamic TCPs. However, these four techniques are highly representative of dynamic techniques and have been widely used in TCP evaluation [16], [46], [56], and statement-level coverage has been shown to be *at least as effective* as other coverage types [46].

6 LESSONS LEARNED

In this section we comment on the lessons learned from this study and their potential impact on future research:

Lesson 1. Our study illustrates that different test granularities impact the effectiveness of TCP techniques, and that the finer, method-level, granularity achieves better performance

in terms of APFD and APFDc, detecting regression faults more quickly. This finding should encourage researchers and practitioners to use method-level granularity, and perhaps explore even finer granularities for regression test-case prioritization. Additionally, researchers should evaluate their newly proposed approaches on different test granularities to better understand the effectiveness of new approaches. Moreover, APFDc values are relatively less consistent with APFD values at test-method level and vary more dramatically as compared to APFD values. This suggests that researchers should evaluate novel TCP approaches in terms of different types of metrics to better investigate the approaches' effectiveness.

Lesson 2. The performance of different TCPs varies across different subject programs. One technique may perform better on some subjects but perform worse on other subjects. For example, TP_{topic} performs better than TP_{cg-add} on *webbit*, but performs worse than TP_{cg-add} on *wsc*. This finding suggests that the characteristics of each subject are important to finding suitable TCPs. Furthermore, we find that the selection of subject programs and the selection of implementation tools may carry a large impact regarding the results of the evaluation for TCPs (e.g., there can be large variance in the performance of different techniques depending on the subject, particularly for static approaches). This finding illustrates that the researchers need to evaluate their newly proposed techniques on a *large* set of real subject programs to make their evaluation reliable. To facilitate this we provide links to download our subject programs and data at [48]. Additionally, a potential avenue for future research may be an adaptive TCP technique that is able to analyze certain characteristics of a subject program (e.g., complexity, test suite size, libraries used) and modify the prioritization technique to achieve peak performance.

Lesson 3. Our study demonstrates that while TCP techniques tend to perform better on larger programs, subject size does not significantly impact comparative measures of APFD and APFDc between TCP techniques. Thus, when the performance of TCP techniques are compared against each other on either large or small programs, similar results can be expected. This finding illustrates scalability of various TCP techniques. Also, our experimental results show that software evolution does not have clear impact on comparative TCP effectiveness.

Lesson 4. Our study demonstrates that mutant quantity and type selected in the experimental settings for measuring the effectiveness of TCP techniques does not dramatically impact the results in terms of APFD or APFDc metrics. This finding provides practical guidelines for researchers, confirming the comparative validity of the mutant seeding process of prior TCP work, and also provides evidence that the fault quantity and type factors are less important to investigate in future work.

Lesson 5. Our findings illustrate that the studied static and dynamic TCP techniques agree on only a small number of found faults for the top ranked test-methods and classes ranked by the techniques, and the most highly prioritized test cases by different TCP techniques share similar capabilities in detecting different types of mutation faults. This suggests several relevant avenues for future research. For instance, (i) it may be useful to investigate specific TCP

techniques to detect important faults faster when considering the fault severity/importance [20], [38], [65] instead of fault types (e.g., different mutant types) during testing; (ii) differing TCP techniques could be used to target specific types of faults or even faults in specific locations of a program; and (iii) static and dynamic information could *potentially* be combined in order to achieve higher levels of effectiveness. Furthermore, the similarity study performed in this paper has not been a core part of many TCP evaluations, and we assert that such an analysis should be encouraged moving forward. While APFD and APFDc provide a clear picture of the relative effectiveness of techniques, it cannot effectively illustrate the *difference set* of detected faults between two techniques. This is a critical piece of information when attempting to understand new techniques and how they relate to existing research.

7 CONCLUSION

In this work, we perform an extensive study empirically comparing the effectiveness, efficiency, and similarity of detected faults for static and dynamic TCP techniques on 58 real-world Java programs mined from GitHub. The experiments were conducted at both test-method and test-class levels to understand the impact of different test granularities on the effectiveness of TCP techniques. The results indicate that the studied static techniques tend to outperform the studied dynamic techniques at the test-class level in terms of both APFD and APFDc metrics, whereas dynamic techniques tend to outperform the static techniques at test-method level in terms of APFD. APFDc values are generally consistent with APFD values at test-class level but relatively less consistent with APFDc at test-method level. In addition, APFDc values vary more dramatically across all subject programs as compared to APFD values. We also observed that subject size, software evolution, and mutant quantities and types within each faulty group/version do not significantly impact comparative measures of TCP effectiveness. Additionally, we found that the faults uncovered by static and dynamic techniques for the highest prioritized test cases uncover mostly dissimilar faults, which suggests promising avenues for future work. Finally, we found evidence suggesting that different TCP techniques tend to perform differently across subject programs, which suggests that certain program characteristics may be important when considering which type of TCP technique to use.

ACKNOWLEDGMENTS

We would like to thank the anonymous TSE and FSE reviewers for their insightful comments that significantly improved the quality of a previous version of this paper and provided useful suggestions for extending the study as presented in this journal paper submission. This work is supported in part by the NSF CCF-1218129, CCF-1566589, and NSF CNS-1510239 grants. Any opinions, findings, and conclusions expressed herein are the authors' and do not necessarily reflect those of the sponsors. This paper is an extension of "A Large-scale Empirical Comparison of Static and Dynamic Test Case Prioritization Techniques" that appeared in Proceedings of 24th ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE'16), Seattle, WA, November 13-18, 2016, pp. 559-570.

REFERENCES

- [1] Github. [Online]. Available: <https://github.com>
- [2] P. Ammann, M. E. Delamaro, and J. Offutt, "Establishing theoretical minimal sets of mutants," in *Proc. IEEE Int. Conf. Softw. Testing Verification Validation*, 2014, pp. 21–30.
- [3] J. H. Andrews, L. C. Briand, and Y. Labiche, "Is mutation an appropriate tool for testing experiments?" in *Proc. 27th Int. Conf. Softw. Eng.*, 2005, pp. 402–411.
- [4] J. H. Andrews, L. C. Briand, and Y. Labiche, "Is mutation an appropriate tool for testing experiments?" in *Proc. 27th Int. Conf. Softw. Eng.*, 2005, pp. 402–411.
- [5] J. H. Andrews, L. C. Briand, Y. Labiche, and A. S. Namin, "Using mutation analysis for assessing and comparing testing coverage criteria," *IEEE Trans. Softw. Eng.*, vol. 32, no. 8, pp. 608–624, Aug. 2006.
- [6] M. J. Arafeen and H. Do, "Test case prioritization using requirements based clustering," in *Proc. 6th Int. Conf. Softw. Testing Verification Validation*, 2013, pp. 312–321.
- [7] ASM. [Online]. Available: <http://asm.ow2.org/>
- [8] C. Catal and D. Mishra, "Test case prioritization: A systematic mapping study," *Softw. Quality J.*, vol. 21, no. 3, pp. 445–478, 2013.
- [9] T. H. Chen, *Studying software quality using topic models*, PhD diss., 2013.
- [10] T. Y. Chen, H. Leung, and I. K. Mak, "Adaptive random testing," in *Proc. Asian Comput. Sci. Conf.*, 2004, pp. 320–329.
- [11] J. Czerwinka, R. Das, N. Nagappan, A. Tarvo, and A. Teterov, "Crane: Failure prediction, change analysis and test prioritization in practice – experiences from windows," in *Proc. 4th IEEE Int. Conf. Softw. Testing Verification Validation*, 2011, pp. 357–366.
- [12] H. Do, S. Mirarab, L. Tahvildari, and G. Rothermel, "An empirical study of the effect of time constraints on the cost-benefits of regression testing," in *Proc. 16th ACM SIGSOFT Int. Symp. Foundations Softw. Eng.*, 2008, pp. 71–82.
- [13] H. Do and G. Rothermel, "A controlled experiment assessing test case prioritization techniques via mutation faults," in *Proc. 1st IEEE Int. Conf. Softw. Maintenance*, 2005, pp. 411–420.
- [14] H. Do and G. Rothermel, "An empirical study of regression testing techniques incorporating context and lifecycle factors and improved cost-benefit models," in *Proc. Int. Symp. Foundations Softw. Eng.*, 2006, pp. 141–151.
- [15] H. Do and G. Rothermel, "On the use of mutation faults in empirical assessments of test case prioritization techniques," *IEEE Trans. Softw. Eng.*, vol. 32, no. 9, pp. 733–752, Sep. 2006.
- [16] H. Do, G. Rothermel, and A. Kinneer, "Empirical studies of test case prioritization in a JUnit testing environment," in *Proc. 15th Int. Symp. Softw. Rel. Eng.*, 2004, pp. 113–124.
- [17] S. Elbaum, P. Kallakuri, A. Malishevsky, G. Rothermel, and S. Kanduri, "Understanding the effects of changes on the cost-effectiveness of regression testing techniques," *Softw. Testing Verification Rel.*, vol. 13, no. 2, pp. 65–83, 2003.
- [18] S. Elbaum, A. Malishevsky, and G. Rothermel, "Prioritizing test cases for regression testing," in *Proc. ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2000, pp. 102–112.
- [19] S. Elbaum, A. Malishevsky, and G. Rothermel, "Incorporating varying test costs and fault severities into test case prioritization," in *Proc. 23rd Int. Conf. Softw. Eng.*, 2001, pp. 329–338.
- [20] S. Elbaum, A. Malishevsky, and G. Rothermel, "Incorporating varying test costs and fault severities into test case prioritization," in *Proc. 23rd Int. Conf. Softw. Eng.*, 2001, pp. 329–338.
- [21] S. Elbaum, A. G. Malishevsky, and G. Rothermel, "Test case prioritization: A family of empirical studies," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 159–182, Feb. 2002.
- [22] S. Elbaum, G. Rothermel, S. Kanduri, and A. G. Malishevsky, "Selecting a cost-effective test case prioritization technique," *Softw. Quality J.*, vol. 12, no. 3, pp. 185–210, 2004.
- [23] M. G. Epitropakis, S. Yoo, M. Harman, and E. K. Burke, "Empirical evaluation of pareto efficient multi-objective regression test case prioritisation," in *Proc. Int. Symp. Softw. Testing Anal.*, 2015, pp. 234–245.
- [24] M. G. Epitropakis, S. Yoo, M. Harman, and E. K. Burke, "Empirical evaluation of pareto efficient multi-objective regression test case prioritisation," in *Proc. Int. Symp. Softw. Testing Anal.*, 2015, pp. 234–245.
- [25] M. Gligoric, L. Eloussi, and D. Marinov, "Practical regression test selection with dynamic file dependencies," in *Proc. Int. Symp. Softw. Testing Anal.*, 2015, pp. 211–222.
- [26] M. Gligoric, A. Groce, C. Zhang, R. Sharma, M. A. Alipour, and D. Marinov, "Comparing non-adequate test suites using coverage criteria," in *Proc. Int. Symp. Softw. Testing Anal.*, 2013, pp. 302–313.

- [27] D. Hao, L. Zhang, L. Zhang, G. Rothermel, and H. Mei, "A unified test case prioritization approach," *ACM Trans. Softw. Eng. Methodology*, vol. 24, 2014, Art. no. 10.
- [28] C. Henard, M. Papadakis, M. Harman, Y. Jia, and Y. L. Traon, "Comparing white-box and black-box test prioritization," in *Proc. 38th Int. Conf. Softw. Eng.*, 2016, pp. 523–534.
- [29] C. Henard, M. Papadakis, G. Perrouin, J. Klein, P. Heymans, and Y. L. Traon, "Bypassing the combinatorial explosion: Using similarity to generate and prioritize t-wise test configurations for software product lines," *IEEE Trans. Softw. Eng.*, vol. 40, no. 7, pp. 650–670, Jul. 2014.
- [30] M. M. Islam, A. Marchetto, A. Susi, and G. Scanniello, "A multi-objective technique to prioritize test cases based on latent semantic indexing," in *Proc. 16th Eur. Conf. Softw. Maintenance Reengineering*, 2012, pp. 21–30.
- [31] JDT, [Online]. Available: <http://www.eclipse.org/jdt/>
- [32] B. Jiang, Z. Zhang, W. K. Chan, and T. H. Tse, "Adaptive random test case prioritization," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2009, pp. 257–266.
- [33] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proc. 22nd ACM SIGSOFT Int. Symp. Foundations Softw. Eng.*, 2014, pp. 654–665.
- [34] R. Just, G. M. Kapfhammer, and F. Schweiggert, "Do redundant mutants affect the effectiveness and efficiency of mutation analysis?" in *Proc. IEEE 5th Int. Conf. Softw. Testing Verification Validation*, 2012, pp. 720–725.
- [35] G. Kaminski, P. Ammann, and J. Offutt, "Improving logic-based testing," *J. Syst. Softw.*, vol. 86, no. 8, pp. 2002–2012, Aug. 2013.
- [36] G. M. Kapfhammer and M. L. Soffa, "Using coverage effectiveness to evaluate test suite prioritizations," in *Proc. 1st ACM Int. Workshop Empirical Assessment Softw. Eng. Languages Technol.: Held Conjunction with 22nd IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2007, pp. 19–20.
- [37] J. Kasurinen, O. Taipale, and K. Smolander, "Test case selection and prioritization: Risk-based or design-based?" in *Proc. ACM-IEEE Int. Symp. Empirical Softw. Eng. Meas.*, 2010, Art. no. 10.
- [38] R. Kavitha and N. Sureshkumar, "Test case prioritization for regression testing based on severity of fault," *Int. J. Comput. Sci. Eng.*, vol. 2, no. 5, pp. 1462–1466, 2010.
- [39] M. Kintis, M. Papadakis, and N. Malevris, "Evaluating mutation testing alternatives: A collateral experiment," in *Proc. 7th Asia Pacific Softw. Eng. Conf.*, 2010, pp. 300–309.
- [40] B. Korel, G. Koutsogiannakis, and L. Tahat, "Application of system models in regression test suite prioritization," in *IEEE Int. Conf. Softw. Maintenance*, 2008, pp. 247–256.
- [41] B. Korel, L. H. Tahat, and M. Harman, "Test prioritization using system models," in *Proc. 21st IEEE Int. Conf. Softw. Maintenance*, Sep. 2005, pp. 559–568.
- [42] LDA, [Online]. Available: <https://cran.r-project.org/web/packages/lda/>
- [43] Y. Ledru, A. Petrenko, S. Boroday, and N. Mandran, "Prioritizing test cases with string distances," *Automated Softw. Eng.*, vol. 19, no. 1, pp. 65–95, 2012.
- [44] O. Legunsen, F. Hariiri, A. Shi, Y. Lu, L. Zhang, and D. Marinov, "An extensive study of static regression test selection in modern software evolution," in *Proc. 24th ACM SIGSOFT Int. Symp. Foundations Softw. Eng.*, 2016, pp. 583–594.
- [45] Z. Li, M. Harman, and R. Hierons, "Search algorithms for regression test case prioritisation," *IEEE Trans. Softw. Eng.*, vol. 33, no. 4, pp. 225–237, Apr. 2007.
- [46] Y. Lu, Y. Lou, S. Cheng, L. Zhang, D. Hao, Y. Zhou, and L. Zhang, "How does regression test prioritization perform in real-world software evolution?" in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng.*, 2016, pp. 535–546.
- [47] Q. Luo, K. Moran, and D. Poshvanyk, "A large-scale empirical comparison of static and dynamic test case prioritization techniques," in *Proc. 24th ACM SIGSOFT Int. Symp. Foundations Softw. Eng.*, 2016, pp. 559–570.
- [48] Q. Luo, K. Moran, L. Zhang, and D. Poshvanyk, "Tse'17 online appendix," [Online]. Available: <http://www.cs.wm.edu/semeru/data/TSE17-TCPSTUDY/>
- [49] Mallet, [Online]. Available: <http://mallet.cs.umass.edu/>
- [50] H. Mei, D. Hao, L. Zhang, L. Zhang, J. Zhou, and G. Rothermel, "A static approach to prioritizing junit test cases," *IEEE Trans. Softw. Eng.*, vol. 38, no. 6, pp. 1258–1275, Nov. 2012.
- [51] C. D. Nguyen, A. Marchetto, and P. Tonella, "Test case prioritization for audit testing of evolving web services using information retrieval techniques," in *Proc. IEEE Int. Conf. Web Serv.*, 2011, pp. 636–643.
- [52] P. Operators. [Online]. Available: <http://pitest.org/quickstart/mutators/>
- [53] M. Papadakis, C. Henard, M. Harman, Y. Jia, and Y. L. Traon, "Threats to the validity of mutation-based test assessment," in *Proc. 25th Int. Symp. Softw. Testing Anal.*, 2016, pp. 354–365.
- [54] PIT, [Online]. Available: <http://pitest.org/>
- [55] X. Qu, M. B. Cohen, and G. Rothermel, "Configuration-aware regression testing: an empirical study of sampling and prioritization," in *Proc. Int. Symp. Softw. Testing Anal.*, 2008, pp. 75–86.
- [56] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Test case prioritization: An empirical study," in *Proc. IEEE Int. Conf. Softw. Maintenance*, 1999, pp. 179–188.
- [57] G. Rothermel, R. J. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Trans. Soft. Eng.*, vol. 27, no. 10, pp. 929–948, Jan.-Mar. 2001.
- [58] R. K. Saha, L. Zhang, S. Khurshid, and D. E. Perry, "An information retrieval approach for regression test prioritization based on program changes," in *Proc. 37th Int. Conf. Softw. Eng.*, 2015, pp. 268–279.
- [59] P. K. Sen, "Estimates of the regression coefficient based on Kendall's tau," *J. Amer. Statistical Assoc.*, vol. 63, no. 324, pp. 1379–1389, 1968.
- [60] A. M. Smith and G. M. Kapfhammer, "An empirical study of incorporating cost into test suite reduction and prioritization," in *Proc. ACM Symp. Appl. Comput.*, 2009, pp. 461–467.
- [61] A. Srivastava and J. Thiagarajan, "Effectively prioritizing tests in development environment," in *Proc. ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2002, pp. 97–106.
- [62] T. H. test, [Online]. Available: <https://en.wikipedia.org/wiki/Tukey>
- [63] S. W. Thomas, H. Hemmati, A. E. Hassan, and D. Blostein, "Static test case prioritization using topic models," *Empirical Softw. Eng.*, vol. 19, no. 1, pp. 182–212, 2014.
- [64] P. Tonella, P. Avesani, and A. Susi, "Using the case-based ranking methodology for test case prioritization," in *Proc. 22nd IEEE Int. Conf. Softw. Maintenance*, 2006, pp. 123–133.
- [65] S. Varun Kumar and M. Kumar, "Test case prioritization using fault severity," *Int. J. Comput. Sci. Technol.*, vol. 1, no. 1, pp. 67–71, 2010.
- [66] WALA, [Online]. Available: <https://github.com/wala/WALA>
- [67] K. R. Walcott, M. L. Soffa, G. M. Kapfhammer, and R. S. Roos, "Time aware test suite prioritization," in *Proc. Int. Symp. Softw. Testing Anal.*, 2006, pp. 1–11.
- [68] K. R. Walcott, M. L. Soffa, G. M. Kapfhammer, and R. S. Roos, "Timeaware test suite prioritization," in *Proc. Int. Symp. Softw. Testing Anal.*, 2006, pp. 1–12.
- [69] S. Wang, D. Buchmann, S. Ali, A. Gotlieb, D. Pradhan, and M. Liaaen, "Multi-objective test prioritization in software product line testing: An industrial case study," in *Proc. 18th Int. Softw. Product Line Conf.*, 2014, pp. 32–41.
- [70] D. Xu and J. Ding, "Prioritizing state-based aspect tests," in *Proc. 3rd Int. Conf. Softw. Testing Verification Validation*, 2010, pp. 265–274.
- [71] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey," *Softw. Testing Verification Reliability*, vol. 22, no. 2, pp. 67–120, 2012.
- [72] D. You, Z. Chen, B. Xu, B. Luo, and C. Zhang, "An empirical study on the effectiveness of time-aware test case prioritization techniques," in *Proc. ACM Symp. Appl. Comput.*, 2011, pp. 1451–1456.
- [73] L. Zhang, D. Hao, L. Zhang, G. Rothermel, and H. Mei, "Bridging the gap between the total and additional test-case prioritization strategies," in *Proc. Int. Conf. Softw. Eng.*, 2013, pp. 192–201.
- [74] L. Zhang, M. Kim, and S. Khurshid, "Localizing failure-inducing program edits based on spectrum information," in *Proc. 27th IEEE Int. Conf. Softw. Maintenance*, 2011, pp. 23–32.
- [75] L. Zhang, D. Marinov, and S. Khurshid, "Faster mutation testing inspired by test prioritization and reduction," in *Proc. Int. Symp. Softw. Testing Anal.*, 2013, pp. 235–245.
- [76] L. Zhang, J. Zhou, D. Hao, L. Zhang, and H. Mei, "Prioritizing JUnit test cases in absence of coverage information," in *Proc. IEEE Int. Conf. Softw. Maintenance*, 2009, pp. 19–28.



Qi Luo received the bachelor's degree from Beihang University, and the master's degree from Tsinghua University. She is working toward the PhD degree in the Department of Computer Science at the College of William and Mary. She is a member of SEMERU research group and advised by Dr. Denys Poshyvanyk. Her research interests include software engineering, performance testing and regression testing. She has published in several top software engineering conferences including: FSE, ICSE and MSR. She is a student member of the ACM and IEEE.



Kevin Moran received the MS degree from the College of William and Mary, in August 2015. He is working toward the PhD degree in the Computer Science Department, College of William and Mary. He is a member of the SEMERU research group and advised by Dr. Denys Poshyvanyk. His main research interest involves facilitating the processes of software engineering, maintenance, and evolution with a focus on mobile platforms. His thesis focused on improving bug reporting for mobile apps through novel

applications of program analysis techniques. He has published in several top peer-reviewed software engineering venues including: ICSE, ESEC/FSE, ICSME, ICST, and MSR. He was recently recognized as the second-overall winner among graduate students in the ACM Student Research competition at ESEC/FSE15. He is a student member of the IEEE and ACM and has served as an external reviewer for ICSE, ICSME, FSE, APSEC, and SCAM. More information available at <https://www.kpmoran.com>.



Lingming Zhang received the BS degree in computer science from Nanjing University, 2007, the MS degree in computer science from Peking University, 2010, and the PhD degree from the Department of Electrical and Computer Engineering, University of Texas at Austin, in May 2014. He is an assistant professor with the Computer Science Department, University of Texas at Dallas. His research interests lie broadly in software engineering and programming languages, including automated program analysis, testing, debugging, and verification, as well as software evolution and mobile computing. He has authored more than 30 papers in premier software engineering or programming language transactions and conferences, including ICSE, FSE, ISSTA, ASE, POPL, OOPSLA, TSE and TOSEM. He has also served on the program committee or artifact evaluation committee for various international conferences (including ASE, ICST, ICSM, ISSRE, OOPSLA, and ISSTA). His research is being supported by NSF and Google. More information available at: <http://www.utdallas.edu/~lxz144130/>. He is a member of the IEEE.



Denys Poshyvanyk received the MS degree in computer science from the National University of Kyiv-Mohyla Academy, Ukraine, in 2003, the MA degrees in computer science from Wayne State University, in 2006, and the PhD degree in computer science from Wayne State University, in 2008. He is the Class of 1953 Term associate professor of Computer Science with the College of William and Mary in Virginia. He served as a program co-chair for ICSME'16, ICPC'13, WCRE'12 and WCRE'11. He currently serves on

the editorial board of the *IEEE Transactions on Software Engineering (TSE)*, the *Empirical Software Engineering Journal (EMSE, Springer)* and *Journal of the Software: Evolution and Process (JSEP, Wiley)*. His research interests include software engineering, software maintenance and evolution, program comprehension, reverse engineering, software repository mining, source code analysis and metrics. His research papers received several Best Paper Awards at ICPC'06, ICPC'07, ICSM'10, SCAM'10, ICSM'13 and ACM SIGSOFT Distinguished Paper Awards at ASE'13, ICSE'15, ESEC/FSE'15, ICPC'16 and ASE'17. He also received the Most Influential Paper Awards at ICSME'16 and ICPC'17. He is a recipient of the NSF CAREER award (2013). He is a member of the IEEE and ACM. More information available at: <http://www.cs.wm.edu/denys/>.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.